

Parallel Computing Note

Name :

Roll No:

Semester:

Contact :

College :

Index

Unit	Title	Page No.
1	Parallel Programming	2
2	Message Passing Programming	16
3	Shared Memory Programming	19
4	Parallel Algorithms I	31
5	Parallel Algorithms II	40
–	Some Missing Topics	49

Unit 1

Parallel Programming

Parallel Programming:

The use of multiple processors together to solve a problem faster and more efficiently is called parallel programming. Instead of solving a problem step by step on a single processor, parallel programming breaks a large problem into smaller tasks and distributes them across multiple processors, which all execute at the same time.

Some key points are:

-  **Core Idea:** It uses multiple processors simultaneously to solve a single problem, reducing the time needed to complete it.
-  **How it works:** A problem is broken down into smaller steps. Instructions are delivered to multiple processors, and they all execute their part of the solution at the same time.
-  **Compared to Concurrent Programming:** Parallel programming produces the same results as concurrent programming, but does so in less time and with greater efficiency.
-  **Everyday Use:** Most modern computers, including laptops and personal desktops, use parallel programming in their hardware to complete background tasks quickly and smoothly.
-  **Wide Application:** It is used across many fields, including science, engineering, retail, and research.
-  **Most Common Societal Use:** From a societal perspective, it is most widely seen in web search engines, applications, and multimedia technologies.

Data Parallelism:

Data parallelism is a type of parallelism where the same operation is performed on multiple pieces of data at the same time, across multiple processors. Here, data is distributed and processed across multiple processors simultaneously in a parallel computing environment. Some key points are:

-  **How it works:** Instead of one processor handling all the data, the data is split and distributed across different nodes (processors), and each node works on its own portion at the same time.
 -  **Common Data Structures:** It works very well with regular data structures like arrays and matrices, where each element can be processed independently and in parallel.
1. **In SIMD Systems:** In a multiprocessor system using **SIMD (Single Instruction, Multiple Data)**, data parallelism is achieved when:
 - ❖ All processors execute the same instruction
 - ❖ But each processor works on different portions of the distributed data
 - ❖ This allows massive amounts of data to be processed very quickly

Function or Task Parallelism:

[Control Parallelism]

Task Parallelism is a type of parallelism where different tasks are executed simultaneously across multiple processors. Here, different tasks or functions are distributed across multiple processors, where each processor works on different task at the same time. A common example of Task parallelism is Pipelining.

Some key points are:

-  **How it works:** Instead of splitting data, task parallelism splits the work (tasks/threads) and assigns different tasks to different processors, which run them concurrently.

- 🚦 **In Multiprocessor Systems:** Task parallelism is achieved when each processor executes a different thread or process, either on the same data or on different data.
- 🚦 **Focus on Processing, not Data:** Unlike data parallelism which focuses on distributing data, task parallelism focuses on distributing the processing work (threads and tasks) among processors.
- 🚦 **Real World Balance:** In practical, most real programs are not purely data parallelism or task parallelism. They fall somewhere in between task parallelism and data parallelism, using a mix of both.

Difference Between:

Data Parallelism	Task Parallelism
➤ Same task run at the same time	➤ Different tasks run at the same time
➤ Same data is distributed across processors.	➤ Same data may or may not be distributed.
➤ Each processor executes the same instruction.	➤ Each processor executes a different thread.
➤ Focuses on distributing the data.	➤ Focuses on distributing the work/tasks.
➤ Easier to manage and implement	➤ Harder to manage and implement
➤ Used when data can be processed independently.	➤ Used when tasks are independent of each other.
➤ Example: Matrix processing	➤ Example: Pipelining

Pipelining:

Pipelining is a technique where multiple instructions are overlapped and executed simultaneously by passing them through a series of stages. It is also called pipelining processing. Pipelining significantly increases the overall instruction throughput, meaning more instructions are executed in less time.

Pipelining Works by overlapping multiple instructions during execution. Instead of waiting for one instruction to finish (before starting the next one), a new instruction begins as soon as the previous one moves to the next stage. The pipeline is divided into multiple stages, connected to each other in a pipe-like structure. Instructions enter from one end and exit from the other end, moving through each stage one by one. The output of each stage is passed as input to the next stage's register.

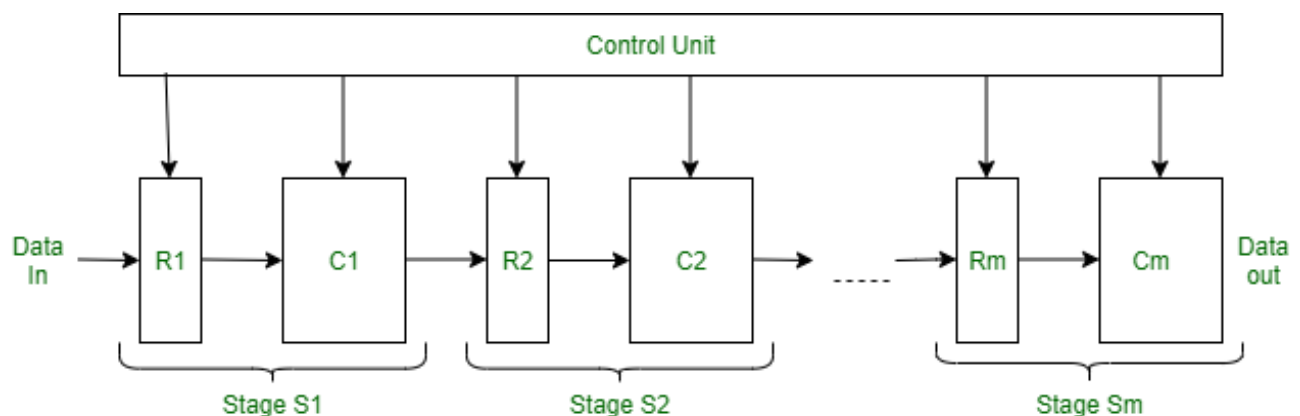


Fig: Pipelining Concept

The components in Pipelining are:

Component	What it does
Data In	Raw data or instruction that enters the pipeline
R1, R2 ... Rm	Input Registers: temporarily hold/store the data at each stage
C1, C2 ... Cm	Combinational Circuits: perform the actual operation on the data
S1, S2 ... Sm	Stages: each stage is one complete unit of Register + Circuit
Control Unit	Controls and coordinates all stages of the pipeline
Data Out	Final result that comes out after passing through all stages

Flynn's Taxonomy:

Flynn's Taxonomy is a classification system for computer architectures, proposed by Michael J. Flynn in 1966. It is widely used in the design of modern processors and their functionalities. Flynn classified all computer architectures using 2 simple questions. They are:

1. How many instructions streams are running? *[Either Single (S) or Multiple (M)]*
2. How many data streams are being processed? *[Either Single (S) or Multiple (M)]*

By combining the answers to these questions, we get 4 possible combinations which forms 4 classifications as:

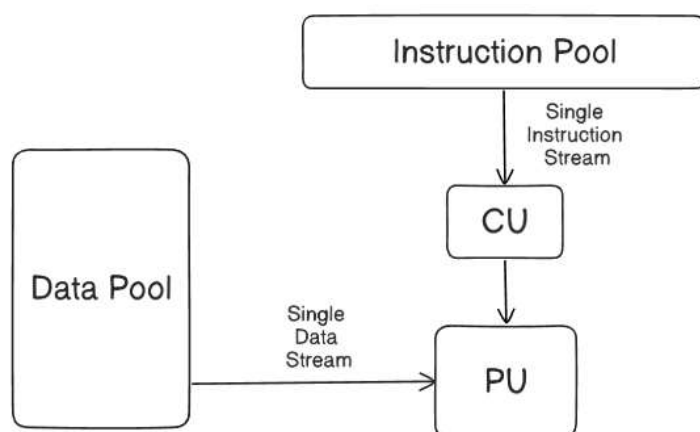
1. Single Instruction Stream, Single Data Stream (**SISD**)
2. Single Instruction Stream, Multiple Data Stream (**SIMD**)
3. Multiple Instruction Stream, Single Data Stream (**MISD**)
4. Multiple Instruction Stream, Multiple Data Stream (**MIMD**)

Single Instruction Stream, Single Data Stream (SISD):

SISD is the simplest and most traditional computer architecture where only one instruction is processed on one piece of data at a time i.e. there is no parallelism at all.

The Processing Unit (PU) is the main processor that handles all operations. The CU is responsible for fetching one instruction at a time from the Instruction Pool, reading it, and executing it before moving to the next. After fetching the instruction, the CU generates control signals to direct the PU to operate on a single Data Stream (DS) fetched from the Data Pool.

Common examples of SISD architecture include older personal computers (PCs) with a single processor.



(Note: By 2010, most PCs had evolved to have multiple cores, moving away from pure SISD).

Instruction Pool: Memory where all instructions are stored

Data Pool: Memory where all data is stored

Fig: SISD

Single Instruction Stream, Multiple Data Stream (SIMD):

SIMD is a computer architecture where one single instruction is applied to multiple different pieces of data at the same time, making it a powerful form of parallel computing.

The Processing Unit (PU) is the main processor that handles all operations. In SIMD, the CU fetches a single instruction from the Instruction Pool and broadcasts that same instruction to multiple Processing Units (PUs), each of which operates on a different piece of data from the Data Pool – all at the same time. Instructions can be executed either sequentially using techniques like pipelining, or in parallel using multiple functional units working simultaneously.

A closely related concept to SIMD is SIMT (Single Instruction, Multiple Threads). It is simply a combination of SIMD and multithreading, meaning one instruction runs across many threads at the same time. SIMT is not a separate category in Flynn's Taxonomy – It is just a subset of SIMD. The term was popularized by Nvidia to describe how their GPU architecture works.

Example: graphics processing units (GPUs)

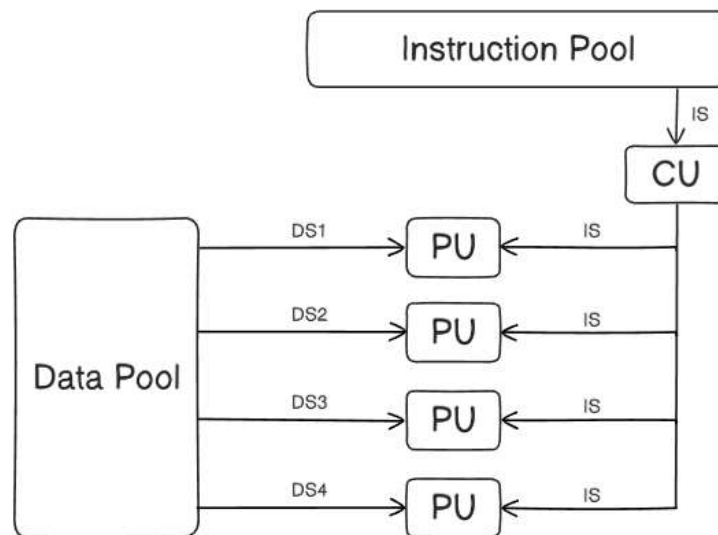


Fig: SIMD

Multiple Instructions Streams, Single Data Stream (MISD):

MISD is a computer architecture where multiple different instructions operate on the same single piece of data at the same time. Unlike SIMD where one instruction works on many data, MISD does the opposite – many instructions work on just one piece of data.

In MISD, there are multiple Processing Units (PUs), and each PU has its own Control Unit (CU). Each CU fetches a different instruction from its own Instruction Pool and generates its own control signals – but all PUs receive and operate on the same single data stream from one shared Data Pool. This means each PU is independently processing the same data in a different way, at the same time.

This is a very uncommon architecture and is generally not used for everyday computing. Instead, it is mostly used for fault tolerance, where multiple processing units perform different operations on the same data and must all agree on the result - If one unit produces a wrong result, the others can detect and correct the error. This makes MISD extremely useful in critical and high-risk systems where errors can be catastrophic.

A well-known real-world example of MISD is the Space Shuttle flight control computer, where multiple systems processed the same data independently to ensure accuracy and safety.

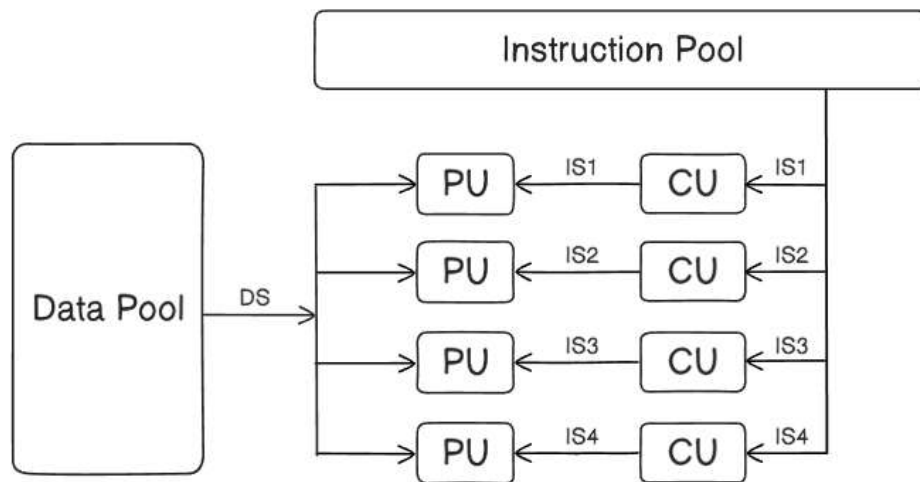


Fig: MISD

Multi Instruction streams, Multi Data Streams (MIMD):

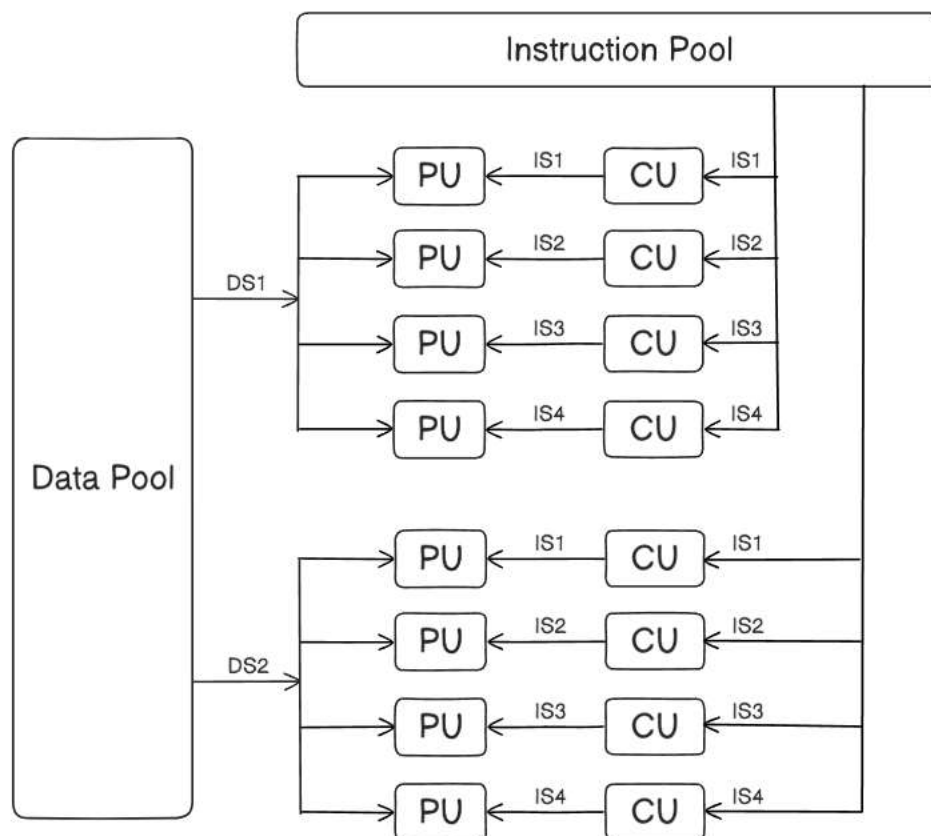


Fig: MIMD

MIMD is a computer architecture where multiple autonomous processors simultaneously execute different instructions on different data at the same time. Unlike MISD where many instructions work on the same data, MIMD does the opposite – many instructions work on many different pieces of data, making it the most powerful and flexible of all four Flynn classifications.

In MIMD, there are multiple Processing Units (PUs), and each PU has its own Control Unit (CU). Each CU independently fetches a different instruction from Instruction Pool and generates control signals and at the same time, each PU also receives a different piece of data from its own Data Pool. This means every PU is completely autonomous, processing different data in a different way, all at the same time, with no dependency on each other.

MIMD architectures support two types of memory organization. In a shared memory system, all processors access one common memory space, making communication between processors simple and fast. In a distributed memory system, each processor has its own private memory and processors communicate by passing messages to one another.

A common real-world example include multi-core processor, such as Intel Core i3, Intel Core i7, AMD Ryzen 5, etc. These are found in everyday modern computers where each core independently executes different instructions on different data at the same time.

Parallel Algorithm Design:

A parallel algorithm is a set of tasks or processes that execute simultaneously and communicate with each other to solve a given problem. A task is simply a part of a program that runs on a processor.

Parallel algorithm design is about how these tasks are broken down, organized, and coordinated to work together efficiently at the same time. A common model used to represent this is the Task/Channel model, where tasks communicate through channels to exchange data.

Model:

A model is a computation or formula that is formed as a result of an algorithm. It takes some values as input, processes them, and produces some value as output.

Task/ Channel Model:

It is a way of viewing a parallel program as a collection of tasks that communicate with each other by sending messages through channels. This model can be represented as a graph where each node represents a task and each edge represents a channel – showing how tasks are connected and how data flows between them.

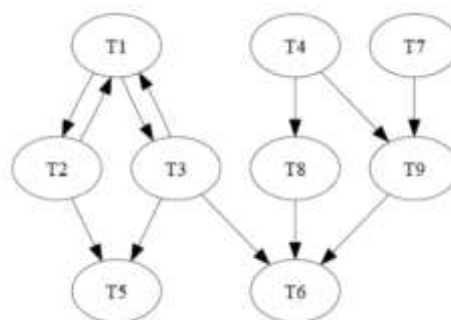


Fig: A parallel computation viewed as a task/ channel graph

Task:

A task is independent executable unit consisting of three things:

- 🚦 **Executable unit:** Actual code the task runs
- 🚦 **Local memory:** Its own private memory that only it can access. Accessing this is called a **local data access**
- 🚦 **I/O Ports:** Entry and exit points used for sending and receiving data to and from other tasks

Channel:

A channel is a message pipe that connects one task's output port to another task's input port. It is the pathway through which tasks send and receive messages. Any data sent or received through a channel is called a nonlocal data access, because the data is coming from or going to another task outside of its own local memory.

Foster's Design Methodology:

Foster's Design Methodology is a method used to design parallel programs (especially for distributed-memory systems) in a structured way. It consists of four steps:

1. Partitioning
2. Communication
3. Agglomeration
4. Mapping.

Partitioning:

Partitioning is the first step in Foster's design methodology where a large problem is divided into many smaller tasks called primitive tasks. The main goal of this step is to maximize parallelism so that multiple tasks can run simultaneously on different processors. This is the only stage where parallelism is increased; later stages usually reduce it.

Partitioning can be done in two ways. In domain decomposition, the data is split into smaller pieces and each piece is assigned to a separate task. In functional decomposition, the focus is on splitting the work rather than the data: different operations that can run at the same time are identified and each is assigned as its own task. A good partitioning strategy ensures that tasks are efficient, balanced, and scalable.

The partitioning checklist are given as:

- ✚ Tasks are much more than processors
- ✚ Redundant computation and data are minimized
- ✚ Tasks are roughly equal in size
- ✚ Number of tasks increases with problem size

This ensure that we create enough balanced work for all processors without wasting resources. It helps achieve better parallel performance and scalability.

Communication:

After partitioning, the next step is communication, where we determine how tasks exchange data with each other. Since tasks often depend on results from other tasks, communication becomes necessary. It can be local communication, where tasks interact with nearby tasks, or global communication, where one task communicates with all others.

The goal is to minimize communication because excessive communication slows down the system. Efficient communication design improves overall performance and avoids bottlenecks. The communication checklist are:

- ✚ Communication is balanced among tasks
- ✚ Each task communicates with a small number of tasks
- ✚ Communication can occur simultaneously (parallel)
- ✚ Computation can occur simultaneously

These ensure communication is efficient and not a bottleneck. Too much or uneven communication can slow down the system, so balance and simplicity are key.

Agglomeration:

Agglomeration is the step where many small primitive tasks are grouped together into larger tasks to make execution more efficient. While partitioning creates as many tasks as possible, having too many small tasks increases communication overhead. Agglomeration solves this by combining related tasks into one, reducing communication cost, improving data locality, and making the system more practical – while still maintaining enough parallelism for efficient execution.

The agglomeration checklist are:

-  **Locality is maximized:** tasks that share data are grouped together so data does not have to travel far, making execution faster
-  **Replicated computation is less costly than communication:** repeating a small calculation inside a task is acceptable if it saves more time than sending the result to another task through communication
-  **Replicated data is small:** any data that is copied and shared across tasks must be kept small enough so the algorithm can still handle large problems without running out of memory
-  **Tasks are balanced:** all tasks should have a roughly equal amount of work and communication so no single task is overloaded while others sit idle
-  **Number of tasks grows with problem size:** as the problem gets bigger, the number of tasks should increase naturally to handle the extra work
-  **Number of tasks $\geq$ number of processors:** there must always be at least as many tasks as there are processors so no processor sits idle
-  **Changes to sequential code are minimal:** the modifications made to the original sequential code to make it parallel should be as small as possible

These points ensure that combining tasks actually **improves performance instead of harming it**. It balances computation and communication while keeping the system scalable.

Mapping:

Mapping is the final step where tasks are assigned to processors for execution. The goal is to distribute tasks in such a way that all processors are efficiently utilized and no processor remains idle. Proper mapping also reduces communication cost by assigning related tasks to nearby processors. This step is especially important in distributed memory systems where communication is expensive, while in shared memory systems it is often handled automatically.

The mapping checklist are:

-  Workload is evenly distributed
-  Communication cost is minimized
-  All processors are utilized efficiently

There ensure efficient use of processors and faster execution. Poor mapping can lead to idle processors or excessive communication delays.

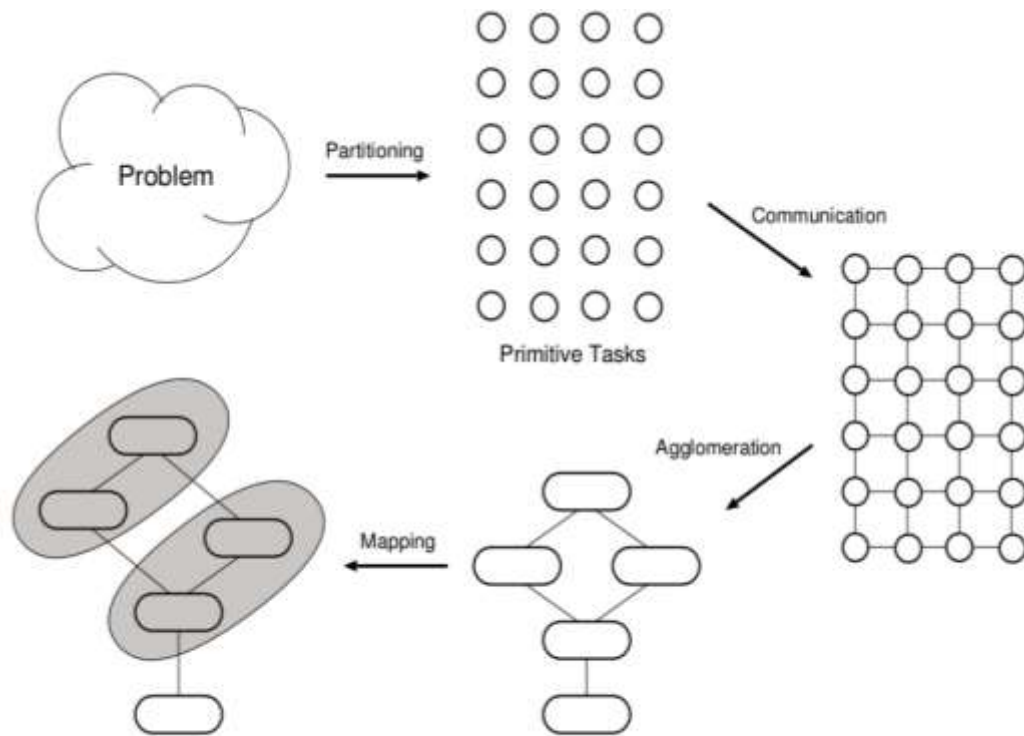


Fig: Foster's design methodology graphical representation.

Boundary Value Problem (BVP):

[Yesko numerical ahila samma tw aako xaina !!]

A BVP is a type of differential equation problem where solution is required to satisfy conditions at 2 different points (called boundaries). In other words, instead of giving all conditions at one point (like in initial value problems), BVP specifies values at the start and end points of the interval.

Example:

$$\frac{d^2y}{dx^2} = f(x), \quad y(a) = A, \quad y(b) = B$$

Here, the value of y are given at 2 different points a and b, so it is a BVP.

There are different methods to solve boundary Value Problems. The most common ones are:

- | | | |
|-----------------------------|-------------------------|-----------------|
| 1. Shooting Method | [Numerical Afai padhne] | [Aako tw xaina] |
| 2. Finite difference Method | [Numerical Afai padhne] | [Aako tw xaina] |

Speedup:

Speedup is a measure of how much faster a parallel program runs compared to a sequential program. It tells you the benefit gained by using multiple processors instead of one. The formula for speedup is:

$$\text{Speedup} = \frac{\text{Time taken by sequential program}}{\text{Time taken by parallel program}}$$

For example: If a sequential program takes 100 seconds and the parallel program takes 25 seconds, then:

$$\text{Speedup} = \frac{100}{25} = 4$$

This means the parallel program is **4 times faster** than the sequential one.

Efficiency:

Efficiency measures how well the processors are being used. A speedup of 4 using 4 processors is perfect — but in reality, processors are never used 100% of the time due to communication and waiting.

Efficiency tells you how effectively each processor is being utilized. The formula for efficiency is:

$$Efficiency = \frac{Speedup}{Number\ of\ Processors}$$

For example:

If speedup is 4 and number of processors is 4:

$$Efficiency = \frac{4}{4} = 1 \text{ (or 100\%)}$$

If speedup is 3 with 4 processors:

$$Efficiency = \frac{3}{4} = 0.75 \text{ (or 75\%)}$$

Amdahl's Law:

Amdahl's Law is a principle in computer architecture used to estimate the speedup of a program when multiple processors are used. It is mainly applied in parallel computing to understand how performance improves with additional processing resources.

It states that, the maximum speedup achieved by parallel processing is limited by the part of the program that cannot be parallelized. It helps to determine how much faster a program can run when multiple processors are used. The formula for Amdahl's law is:

I.e.

$$Speedup = \frac{1}{(1 - P) + \frac{P}{N}}$$

(1 - P) → Time for Non-parallel portion
P/N → Time for Parallel portion

where,

- **P** = Parallelizable portion of the program
- **(1 - P)** = Non-parallelizable portion of the program
- **N** = Number of processors

Example: If 75% of a program is parallel and we use 4 processors. Then speedup is:

$$Speedup = \frac{1}{(1 - 0.75) + \frac{0.75}{4}} = \frac{1}{0.4375} = 2.28$$

This means even with 4 processors, the speedup is only **2.28x** (not 4x) because the sequential portion always limits the maximum speed up.

Limitations of Amdahl's Law:

1. Amdahl's Law assumes that the problem size remains constant as the number of processors increases. But in practical systems, when more processors are increased, larger problems can be solved.
2. It assumes the sequential portion remains constant and becomes main bottleneck. But in real applications, the parallel portion often grows larger than the sequential portion.

3. It shows limited speedup even after adding many processors. But in practicality, modern supercomputers achieve high performance by increasing the amount of parallel work.
4. It ignores the communication and synchronization time between processors. I.e. Processors spend extra time exchanging data and coordinating tasks.
5. It is not suitable for scalable parallel computing systems because modern systems are designed to handle larger workloads as the number of processors increases.

Gustafson Barsis's Law:

Gustafson Barsis's Law was purposed to overcome the limitations of Amdahl's law. It offers a more optimistic and realistic view of parallel computing.

Gustafson Barsis's Law states that, "The scaled speedup of a parallel program increases linearly with the number of processors, provided that problem size is increased along with the number of processors while keeping the execution time constant".

In simple terms, Gustafson–Barsis Law means that when we use more processors, we increase the problem size instead of keeping it fixed. Even though the problem becomes larger, the execution time remains almost the same. Because of this, more work is done in the same time, so the speedup increases almost linearly with the number of processors.

The formula for Gustafson Barsis's law is:

$$S(N) = N - (N - 1) * s$$

where,

$S(N)$ = Scaled speedup with N processors

N = Number of Processors

s = Non parallelizable portion of the Program

Example:

Suppose $s = 0.1$ (10% of the program is sequential) and you use 8 processors. Then.

Soln:

$$S(N) = 8 - (8 - 1) * 0.1$$

$$S(N) = 8 - (7 * 0.1)$$

$$S(N) = 8 - 0.7$$

$$S(N) = 7.3$$

This means, with 8 processors, we can achieve a speedup of 7.3x – much more optimistic than what Amdahl's law would predict.

Example:

An application executing on 64 processors using 5% of the total time on non-parallelizable computations. What is the scaled Speedup?

Solution:

Here number of processors (N) = 64

Serial fraction (s) = 5% = $5/100 = 0.05$

Scaled speedup (S) = ?

We know that,

$$S = N + (N - 1) * s$$

$$S = 64 + (64-1) * 0.05 = 60.85$$

Derivation:

We know that,

$$s + p = 1 \quad [\text{serial} + \text{parallel portion}]$$

From Gustafson's assumption – serial part remains constant and parallel part gets distributed across N processors and also we increase problem size. So, total scaled work becomes:

$$S(N) = s + p * N \quad [\text{serial} + \text{parallel work}]$$

$$S(N) = s + (1 - s) * N$$

$$S(N) = s + N - s * N$$

$$S(N) = N - (N - 1) * s$$

Note:

Exam ma Parallel portion of program ni dina sakx, so $(1 - \text{parallel portion})$ garda non-parallel portion aauxa. Keep this in mind.

Note:

In Amdahl's law I have used speedup and in Gustafson Barsis law I have used Scaled Speedup. This is because Amdahl's assumes the problem size remains constant even if no. of processors increases where as Gustafson Basis law, the problem size also increases as no. of processors increases.

Speedup → It measures how much faster a program runs using multiple processors compared one processor. Here, problem size remains same (constant).

Scaled Speedup → It measures how much larger problem can be solved using more processors in the same time.

Karp-Flatt Metric:

The Karp–Flatt Metric is a performance measure used in parallel computing to evaluate how well a program is parallelized. It estimates the effective serial fraction of a program based on its observed speedup. It is represented by 'e'.

For a parallel program running on 'p' processors with observed speedup 'ψ', the Karp–Flatt metric 'e' is given by:

$$e = \frac{\frac{1}{\psi} - \frac{1}{p}}{1 - \frac{1}{p}}$$

ψ is called psi (read as **sai**).

where,

- e = Experimentally determined serial fraction
- ψ = Observed speedup
- p = Number of processors

Interpretation:

1. **Small e (closer to 0)** → Good parallelization, Minimal serial portion
2. **Larger e** → Poor parallel efficiency due to more serial work or overhead.

Note:

🚦 If 'e' stays approximately constant as 'p' increases → Serial portion of the program is the main bottleneck

🚦 If 'e' increases as 'p' increases → Parallel portion of the program is the main bottleneck.

Extra:

"e remains approximately constant as p increases" means that the value of the Karp–Flatt metric changes very little even when more processors are added.

Example:

Benchmarking a parallel program on 2 to 8 processors produces the following speedup results. What is the primary reason for the parallel program achieving a speedup of only 4.71 on 8 processors?

p	2	3	4	5	6	7	8
ψ(n,p)	1.82	2.50	3.08	3.57	4.00	4.38	4.71

Solution

We compute $e(n,p)$ for each processor using the Karp-Flatt formula:

$$e = \frac{\frac{1}{\psi} - \frac{1}{p}}{1 - \frac{1}{p}}$$

Yesko calculator wala formula banayera calculator ma hanne NM subject ma gare jastai, ani fatafat table banaune...

For $p = 2, \psi = 1.82$:

$$e = \frac{\frac{1}{1.82} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0.549 - 0.5}{0.5} = \frac{0.049}{0.5} \approx 0.1$$

Yestai 2, 3 line solve gareko dekhaunu ani similarly bhanera table banai dinu time bachxa !!

For $p = 3, \psi = 2.50$:

$$e = \frac{\frac{1}{2.50} - \frac{1}{3}}{1 - \frac{1}{3}} = \frac{0.4 - 0.333}{0.667} = \frac{0.067}{0.667} \approx 0.1$$

Similarly, calculating e we, get following values.

p	2	3	4	5	6	7	8
$\psi(n, p)$	1.82	2.50	3.08	3.57	4.00	4.38	4.71
$e(n, p)$	0.1	0.1	0.1	0.1	0.1	0.1	0.1

Conclusion:

Since, ' e ' remains constant at 0.1 for all processor and does not increase as more processors are added, the main limitation to speedup is the **10% sequential portion** of the program that cannot be parallelized.

This indicates that the reduced speedup is caused by the inherent serial part of the computation rather than by parallel overhead.

If parallel overhead (such as communication or synchronization cost) were the major issue, the value of e would increase as the number of processors increased.

Iso-efficiency Metric:

The Iso-efficiency Metric is used in parallel computing to measure the scalability of a parallel system. It describes the relationship between the problem size and the number of processors required to maintain constant efficiency.

As the number of processors increases, parallel overhead such as communication and synchronization also increases. To keep the processors efficiently utilized, the total workload must increase accordingly so that each processor continues to perform enough useful computation. The Iso-efficiency Metric indicates how much the problem size must grow to maintain the same level of efficiency.

Iso-efficiency Function

The Iso-efficiency Function mathematically defines the relationship between the problem size W and the number of processors p required to maintain a fixed efficiency.

It shows how rapidly the workload must increase as processors increase. A smaller iso-efficiency function indicates better scalability because the system requires a smaller increase in problem size to maintain efficiency.

Scalability

A scalable parallel system is one that is able to **increase its performance** as the number of processors increases. A good parallel system should maintain its efficiency even as more processors are added.

Efficiency

Efficiency measures how well the processors are being used. A scalable system should maintain efficiency as the number of processors increases — if efficiency drops significantly when more processors are added, the system is not well scalable.

Unit 2

Message Passing Programming

Message Passing:

Message passing is a technique used for communication between multiple processes running on a computer. Instead of processes being directly connected to each other, they communicate by sending and receiving messages through a message queue.

How Message Passing Works?

In the message passing model:

- Multiple processes can read and write data to a shared message queue without being directly connected to each other
- Messages are stored in the queue until the recipient process retrieves them
- Message queues are widely used for inter-process communication and are supported by most operating systems

In diagram below, both processes P1 and P2 can access the message queue to store and retrieve data.

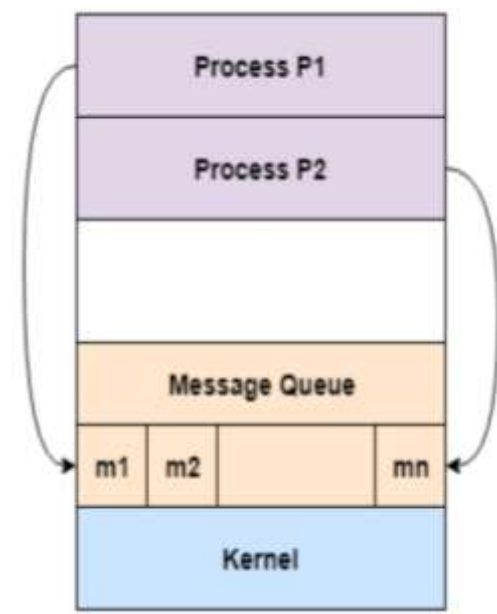


Fig: Message Passing Model

Advantages of Message Passing:

The message passing model is much easier to implement than the shared memory model. It is also easier to build parallel hardware using this model as it is tolerant of higher communication latencies.

Disadvantages of Message Passing:

The message passing model has slower communication than the shared memory model because the connection setup between processes takes time before any data can be exchanged.

Message Passing Interface (MPI):

Message Passing Interface (MPI) is a standardized way of exchanging messages between multiple computers running a parallel program across a distributed memory system.

In parallel computing, multiple computers or even multiple processor cores within the same computer are called nodes. Each node works on a portion of the overall problem, and the main challenge is to keep all nodes working together efficiently. This involves synchronizing the actions of each node, exchanging data between them, and providing command and control over the entire parallel cluster.

MPI solves all of these challenges by providing a standard suite of functions that handle all communication and coordination between nodes – making it the most widely used standard for parallel programming in distributed memory systems.

MPI Standard:

[IMP QN]

The MPI Standard is a message passing library standard developed by the MPI Forum, which has over 40 participating organizations including vendors, researchers, software library developers, and users.

The MPI standard includes:

- ✚ **Point-to-point communication** → for sending and receiving messages between two processes
- ✚ **Collective communication** → for communication among a group of processes
- ✚ **Process management** → for initializing, finalizing, and managing MPI processes
- ✚ **Communication groups and communicators** → for defining and managing groups of processes for communication
- ✚ **Data types** → provides support for predefined and user-defined data types for message passing
- ✚ **Virtual topologies** → Logical arrangement of processes in structures like grids or graphs
- ✚ **Synchronization** → Coordinating processes to ensure correct execution order
- ✚ **Environmental management** → Handling runtime environment information of MPI programs
- ✚ **Error handling** → Detecting and managing communication errors
- ✚ **Attributes and contexts** → Providing additional information and isolation for communication operations

The MPI standard doesn't include:

1. No hardware specification – MPI doesn't depend on specific machines or architectures
2. No automatic parallelization – Programmer must explicitly write parallel code
3. No scheduling or load balancing rules – It doesn't decide how tasks should be distributed efficiently.
4. No debugging tools – MPI doesn't include debugging tools.
5. MPI only provides communication tools – not algorithms.

Basic Concepts of MPI:

1. MPI_Init

Initializes the MPI execution environment. This must always be the first MPI function called before any other MPI operations can be performed.

2. MPI_Comm_size

Retrieves the total number of processes involved in a communicator. It is often used together with MPI_Comm_rank to determine how much concurrency is available for a program.

3. MPI_Comm_rank

Retrieves the rank (ID) of the calling process within the communicator. Each process is assigned a unique rank in the range of 0 to size-1, where size is obtained from MPI_Comm_size.

4. MPI_Send

Performs a standard send operation — it sends a message and returns only when the send buffer can be safely reused.

5. MPI_Recv

Performs a receive operation — it does not return until a matching message has been successfully received.

6. MPI_Finalize

Terminates the MPI execution environment. This must always be the last MPI function called to cleanly shut down all MPI operations.

Timing the MPI Programs:

Two functions are used to time MPI programs:

1. MPI_Wtime

Returns the **elapsed time** on the calling processor as a double precision value. It is used to measure how long a section of code takes to run.

2. MPI_Wtick

Returns the **resolution of MPI_Wtime** in seconds — meaning it tells you the time between two successive clock ticks. For example, if the hardware clock is incremented every millisecond, MPI_Wtick returns 10^{-3} .

The Sieve of Eratosthenes:

The Sieve of Eratosthenes is an ancient algorithm for finding all prime numbers up to a given limit. It works by iteratively marking the multiples of each prime number as composite (not prime), starting from the first prime number 2. It is one of the most efficient ways to find all smaller prime numbers.

Algorithm:

1. Create a list of consecutive integers from **2 to n**: (2, 3, 4, ... n)
2. Start with **p = 2** (the smallest prime number)
3. If $p * p > n$, then stop. *[This condition makes the algo. optimized]*
Else, Mark all **multiples of p** (2p, 3p, 4p, ...) as composite. The number **p itself is not marked**
4. Find the **next unmarked number** greater than p. If no such number exists, **stop**. Otherwise set p to this new number and **go to step 3**.
5. When the algorithm finishes, all **unmarked numbers are prime**

Example: To find all the prime numbers less than or equal to 30, proceed as follows.

1. List all integers from 2 to 30:
2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
2. For P = 2, Cross out all multiples of 2:
2 3 **4** 5 **6** 7 **8** 9 **10** 11 **12** 13 **14** 15 **16** 17 **18** 19 **20** 21 **22** 23 **24** 25 **26** 27 **28** 29 **30**
3. For P = 3, Cross out all multiples of 3:
2 3 5 7 **9** 11 13 **15** 17 19 **21** 23 25 **27** 29

4. For $P = 5$, Cross out all multiple of 5:
2 3 5 7 11 13 17 19 23 **25** 29

5. For $P = 7$, Cross out all the multiple of 7:
2 3 5 7 11 13 17 19 23 29

Since, $p * p = 7 * 7 = 49 > n = 30$. So, we stop here.

So, prime numbers up to 30 are: 2 3 5 7 11 13 17 19 23 29.

Floyd's Warshall Algorithm:

The Floyd-Warshall Algorithm (also known as Floyd's Algorithm) is an algorithm used for finding the shortest paths between all pairs of nodes in a directed weighted graph. It works with both positive and negative edge weights, but does not work if the graph contains negative cycles.

Weighted Graph: A weighted graph is a graph where each edge has a value (weight) assigned to it. This weight typically represents the cost, distance, or time required to travel from one node to another.

Working of Floyds Algorithm:

The algorithm works by comparing all possible paths between every pair of nodes and finding the shortest one. It does this by checking whether going through an intermediate node makes the path shorter.

The key idea is:

"Is the path from node A to node B shorter if we go through node K?"

If yes – update the shortest path.

If no – keep the current path.

This is repeated for every possible intermediate node until all shortest paths are found.

Formula:

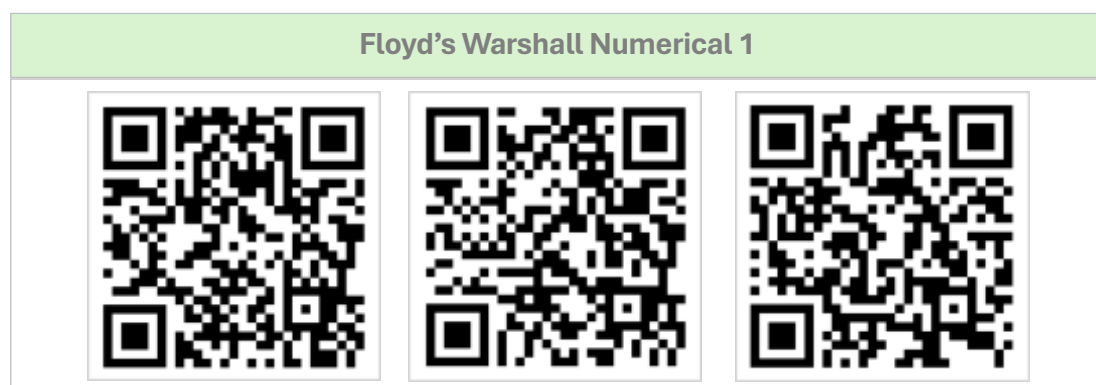
$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

where,

- $D[i][j]$ = Current shortest distance from node i to node j
- $D[i][k]$ = Distance from node i to intermediate node k
- $D[k][j]$ = Distance from intermediate node k to node j

Note:

Exam ma Yo topic ko numerical frequently sodeko xa, so tala diyeko QR wala Video hernu.....



Matrix Vector Multiplication:

Matrix-Vector Multiplication is a operation in linear algebra where a matrix is multiplied by a vector to produce a new vector.

Basic Rules are:

- The number of columns in the matrix must equal the number of elements in the vector
- The result is always a new vector
- Given an $n \times n$ matrix A and a vector x of length n , their product is a new vector $y = A \cdot x$

Formula:

For each entry i of the result vector y :

$$y[i] = \sum_{j=0}^{n-1} A[i][j] \times x[j]$$

This means each element of the result vector y is computed by multiplying each element in a row of A with the corresponding element in vector x and then summing them all up.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad x = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$y[0] = (1 \times 5) + (2 \times 6) = 5 + 12 = 17$$

$$y[1] = (3 \times 5) + (4 \times 6) = 15 + 24 = 39$$

$$y = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

Unit 3

Shared Memory Programming

Shared Memory Model:

Shared memory is a block of RAM that multiple processes can access simultaneously. Rather than passing messages back and forth, processes communicate by directly reading and writing to the same memory space: making it one of the fastest forms of inter-process communication (IPC). It is widely used inside a single program (Ex: between threads) as well as across multiple programs running at the same time on a multiprocessor system.

How it works?

1. A **single shared memory region** is created in RAM and made accessible to multiple processes
2. Each process can **read from and write to** this memory directly
3. Data written by one process is **immediately visible** to other processes
4. No message queues or connection setup is needed
5. Processes must coordinate access to **avoid writing to the same location simultaneously**

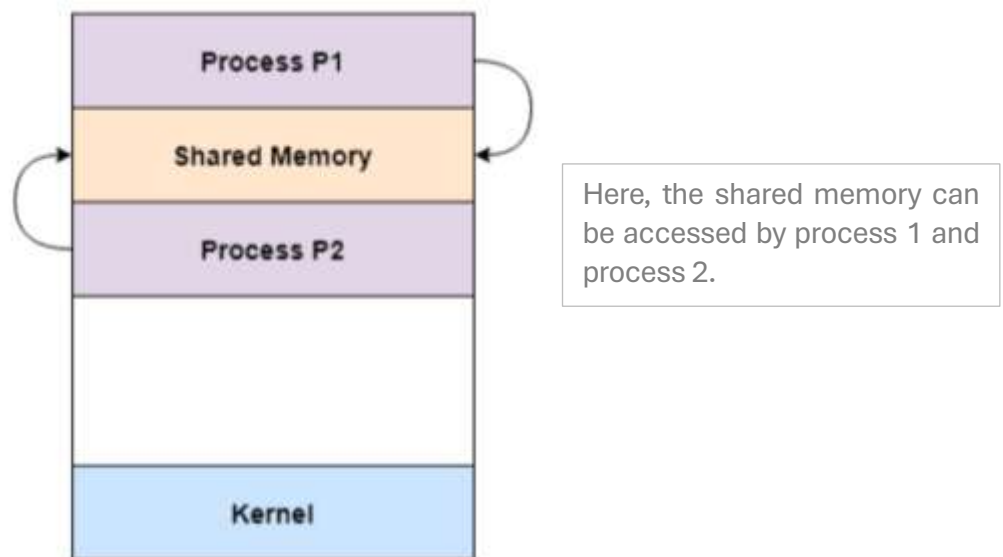


Fig: Shared Memory Model

Advantages of Shared Memory model:

- 🚩 **Fastest IPC method:** direct memory access, no transfer overhead
- 🚩 **No data duplication:** all processes share one memory space
- 🚩 **Great for multithreading:** threads share memory naturally
- 🚩 **Widely supported:** works on POSIX systems and Windows

Disadvantages of Shared Memory model:

- 🚩 **Can cause Race Conditions:** simultaneous writes can corrupt data
- 🚩 **Synchronization required:** needs locks or semaphores to coordinate
- 🚩 **Memory Protection risks:** one process can overwrite another's data
- 🚩 **Single machine only:** doesn't work across networks

Note: POSIX (Portable Operating System Interface) is a set of standards for operating systems. Systems like Linux, macOS, and Unix follow POSIX standards.

OpenMP standard:

OpenMP (Open Multi-Processing) is an API (Application Programming Interface) used to write multi-threaded parallel programs that run on shared memory systems. It allows programmers to explicitly control how tasks are divided and executed across multiple threads. It is supported by major programming languages including C, C++, and Fortran.

The 3 primary components of OpenMP are:

1. Compiler Directives:

Compiler directives appear as comments in the source code and are ignored by compilers unless enabled via a compiler flag. They are used to spawn parallel regions, divide code blocks and loop iterations among threads, serialize sections of code, and synchronize work among threads.

C/C++ Example:

```
#pragma omp parallel default(shared) private(beta, pi)
```

→ This tells the compiler to run the following block in parallel, where all variables are shared except beta and pi which are private to each thread.

2. Runtime Library Routines:

These are built-in functions called during execution to control and monitor threads. They are used to set/query the number of threads and thread IDs, enable/disable dynamic threads and nested parallelism, manage locks, and query wall clock time.

C/ C++ Example:

```
#include <omp.h>
int omp_get_num_threads(void);
```

→ This function **returns the total number of threads** currently active in the parallel region.

3. Environment Variables:

These are system-level settings that control parallel execution at runtime without modifying the code. They are used to set the number of threads, divide loop iterations, bind threads to processors, and control nested parallelism and thread stack size.

C/C++ Example:

```
export OMP_NUM_THREADS=8
```

→ This sets the program to use 8 threads when executing parallel regions.

Parallel For Loops:

A parallel for loop is a loop where iterations run simultaneously across multiple threads instead of one after another. The iterations are divided and distributed among available threads, significantly reducing execution time.

They are a special type of SPMD (Single Program, Multiple Data) programming, meaning the same code runs on multiple threads but with different data. They are commonly used in shared memory models like OpenMP and Intel's Threading Building Blocks (TBB).

Syntax:

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    // loop body
}
```

Example:**parallel_for_loop.c**

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i;
    int a[5], b[5], c[5];

    // Initialize arrays
    for(i = 0; i < 5; i++) {
        b[i] = i;
        c[i] = i * 2;
    }

    // Parallel for loop
    #pragma omp parallel for
    for(i = 0; i < 5; i++) {
        a[i] = b[i] + c[i];
        printf("Thread %d computed a[%d] = %d\n", omp_get_thread_num(), i, a[i]);
    }

    return 0;
}
```

Output

What this does: The array addition loop is split among multiple threads. Each thread computes a portion of the array simultaneously. `omp_get_thread_num()` prints which thread handled each iteration. I.e. output looks something like this: [The thread number may vary each time]

```
Thread 0 computed a[0] = 0
Thread 1 computed a[1] = 3
Thread 0 computed a[2] = 6
Thread 1 computed a[3] = 9
Thread 2 computed a[4] = 12
```

Parallel For Pragma:

A pragma is a compiler directive in C/C++ where the word pragma stands for "pragmatic information". It is a way for the programmer to give special instructions to the compiler. Compilers are generally free to ignore pragmas, but when OpenMP is enabled, they are processed and applied to parallelize the code. In OpenMP, pragmas are used to tell the compiler which parts of the code should run in parallel – without rewriting the entire program.

Syntax:

```
#pragma omp <directive>
```

Example:

```
#pragma omp parallel for          // This makes the for loop parallel
for(int i = 0; i < 10; i++) {
    printf("Hello World From % Thread.\n", omp_get_thread_num());
}
```

Extra:

Without `#pragma omp parallel for`, this loop runs sequentially. With it, OpenMP splits the iterations among available threads and runs them simultaneously.

Private Variables:

In a parallel environment, variables can have either a shared or private context. Variables in shared context are visible to all threads – any thread can read or modify them. Variables in private context are hidden from other threads – each thread gets its own separate copy of the variable, and any changes made to it are not visible to other threads.

When a variable is declared private using the **private(variables)** clause, OpenMP replicates that variable and assigns a local copy to each thread independently.

Syntax:

```
#pragma omp parallel for shared(variables) private(variables)
```

Example:

```
#pragma omp parallel for shared(n, a) private(b)
for(int i = 0; i < n; i++) {
    b = a + i;      // b is private — each thread has its own copy of b
}
```

Critical Section:

In parallel programming, multiple threads often access shared resources simultaneously. Without any control, this can lead to unexpected or incorrect results. A critical section is a protected region of code that ensures only one thread can execute it at a time, preventing conflicts and ensuring data correctness.

Critical sections are essential when dealing with shared resources such as variables, data structures, peripheral devices, or network connections or, anything that cannot safely handle multiple concurrent accesses at the same time.

Why critical sections are needed?

When two processes try to read and write the same variable simultaneously, the result depends on the order in which they execute – which can be unpredictable.

Example:

```
// Process A
y = x + 5;           // reads variable x at time Tx

// Process B
x = 3 + z;           // writes to variable x at time Tx
```

Above code is critical code. If both processes run at the same time, Process A might read the old value of x before B writes, or the new value after – the result is unpredictable.

Above problem can be solved using Critical section. The OpenMP syntax for Critical section is:

```
#pragma omp critical {
    // Critical Code here ...
    // only one thread can execute this at a time
}
```

Example:

```
#pragma omp critical {  
    y = x + 5;  
}  
  
#pragma omp critical {  
    x = 3 + z;  
}
```

Explanation:

Although there are two separate critical blocks in the code, they are **not treated** as independent. In OpenMP, all unnamed `#pragma omp critical` sections share the same lock, so only one process can execute any of them at a time.

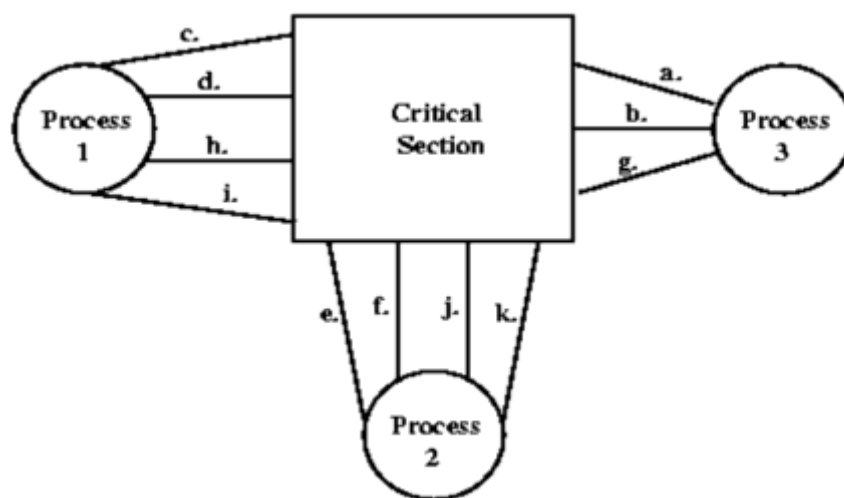


Fig: Representation of the Critical section

Explanation:

The figure shows multiple processes (P1, P2, P3) trying to execute different operations (a, b, c, d, ...) that require access to a common critical section. Although each process has several instructions, all of them must pass through the same critical section, and only one process can execute inside it at a time. The others must wait, ensuring safe and synchronized access to shared resources.

Parallel Reduction:

Parallel reduction is a technique that reduces a collection of elements into a single result using multiple threads working simultaneously. It works by repeatedly halving the number of threads until one final result remains.

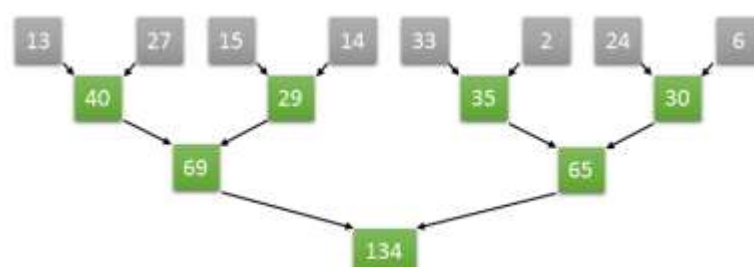


Fig: Parallel Reduction (Sum of an Array)

How it works?

Parallel reduction works by using half the number of threads of the total elements in the dataset. Each thread takes two elements, computes the result (e.g. sum, minimum), and passes it to the next round. The number of threads is then halved and the process repeats until only one final result remains.

Reduction Operator:

A reduction operator is an operator that combines all elements of an array into a single result. It is one of the most commonly used concepts in parallel programming. Reduction operators are associative, meaning the order of grouping does not affect the result, and are often commutative, meaning the order of elements does not affect the result either.

Reduction operators are widely used in programming models such as MapReduce, where the operator is first applied to all elements before they are reduced into a final result. They are also used for broadcasting to distribute data to all processors, and as primary building blocks in complex parallel algorithms.

Example:

```
#pragma omp parallel for reduction(+:sum)
for(int i = 0; i < n; i++) {
    sum += a[i];
}
```

Explanation:

Here each thread independently computes a partial sum of its assigned portion. Once all threads complete, OpenMP automatically combines all partial sums into one final sum. 'reduction' tells OpenMP to perform a reduction operation. '+' operator specifies addition operation. 'sum' is the variable to store the final result.

Each thread gets its own private copy of **sum** starting at 0. Each thread adds its portion of the array to its own private sum. After all threads finish, OpenMP adds all the private sums together into the final sum.

Loop Optimization:

Loop optimization is the process of increasing execution speed and reducing overhead associated with loops. Since most execution time in scientific programs is spent on loops, many optimization techniques have been developed to make them faster. It plays a key role in improving cache performance and making effective use of parallel processing.

The core idea is simple – reducing instructions inside a loop improves overall performance even if some code is added outside it. There are different techniques for loop optimization. They are:

1. Frequency Reduction (Code Motion)
2. Loop Unrolling
3. Loop Jamming

Frequency Reduction (Code Motion):

Code that doesn't change across iterations is moved outside the loop to avoid repeated computation. Use this when an expression does not depend on the loop variable. Do not use it when the expression changes each iteration or moving it affects program behavior.

Example:

Before Optimization	After Optimization
<pre>while(i < 100) { a = Sin(x)/Cos(x) + i; printf("a: %f \n", a); i++; }</pre>	<pre>t = Sin(x)/Cos(x); while(i < 100) { a = t + i; printf("a: %f \n", a); i++; }</pre>

Loop Unrolling:

Loop unrolling removes iterations by writing the loop body multiple times, eliminating loop control overhead. Use when iterations are small and known. Avoid when iterations are large or unknown as it increases code size.

Example:

Before Optimization	After Optimization
<pre>for(int i = 0; i < 5; i++){ printf("Hello\n"); }</pre>	<pre>printf("Hello\n"); printf("Hello\n"); printf("Hello\n"); printf("Hello\n"); printf("Hello\n");</pre>

Loop Jamming:

Loop jamming combines two or more loops with the same range into one, reducing overhead and improving cache efficiency. Use when loops have the same iteration range. Avoid when one loop depends on the result of another.

Example:

Before Optimization	After Optimization
<pre>for(int i = 0; i < 5; i++){ a = i + 5; } for(int i = 0; i < 5; i++){ b = i + 10; }</pre>	<pre>for(int i = 0; i < 5; i++){ a = i + 5; b = i + 10; }</pre>

Q. Types of Parallelism:

There are 2 types of parallelism. They are:

1. Data parallelism [same instruction different data]
2. Task / functional Parallelism [different instruction same data]

Data Parallelism:

Data parallelism (also known as loop-level parallelism) is a form of parallel computing where the same task is performed simultaneously on different portions of data across multiple processors. It is one of the two main forms of parallelism, the other being task parallelism.

In a multiprocessor system, data parallelism is achieved when each processor executes the same instructions but on different pieces of data. Depending on the situation, either a single thread controls all operations, or multiple threads run the same code on different data simultaneously.

How it works?

Consider a 2-processor system (Processor A and Processor B) with a dataset D. Then, Processor A processes the first half of data D and processor B processes the second half of data D. Both execute same instructions on different data simultaneously – reducing overall execution time.

Functional Parallelism:

Functional parallelism (also known as task parallelism or control parallelism) is a form of parallel computing where different tasks are distributed and executed simultaneously across multiple processors or threads.

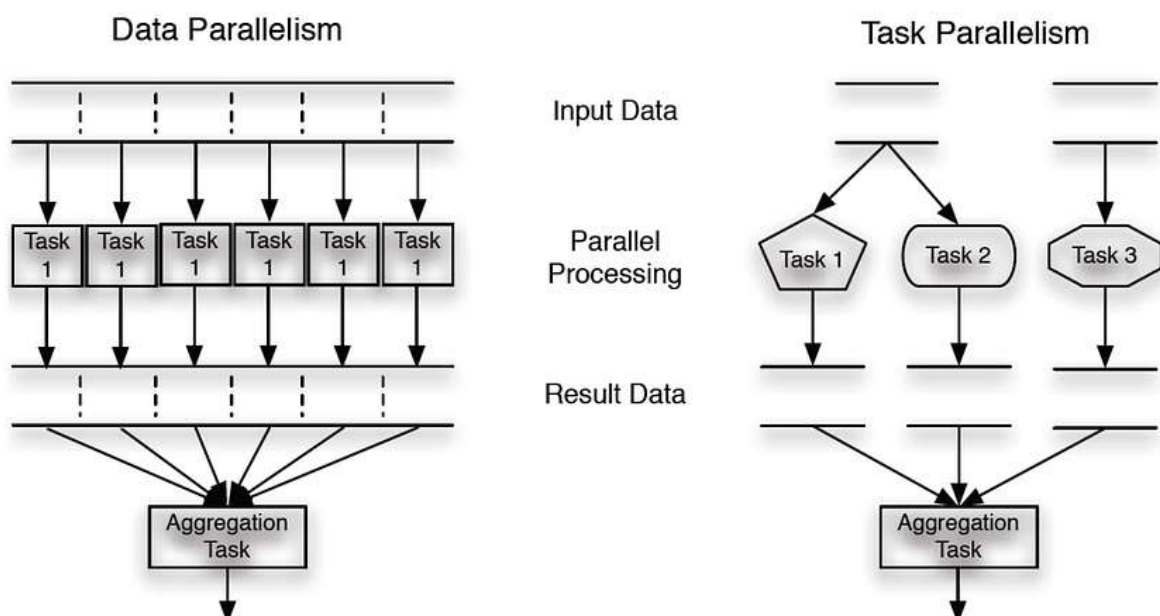
Unlike data parallelism which runs the same task on different data, functional parallelism runs different tasks on the same data at the same time. The focus is on distributing tasks/functions rather than data across processors. A common type of functional parallelism is pipelining — where a single set of data moves through a series of separate tasks, each executing independently of the others, like an assembly line.

Example:

Consider processing an image:

- **Thread 1** → applies brightness filter
- **Thread 2** → applies contrast filter
- **Thread 3** → applies sharpness filter

All three run simultaneously on the same image



Difference Between:

Data Parallelism	Task Parallelism
➤ Same operations performed on different subsets of data	➤ Different operations performed on same or different data
➤ Synchronous computation (<i>all threads execute together in a coordinated way, often waiting at certain points</i>)	➤ Asynchronous computation (<i>threads run independently without waiting, proceed at their own speed</i>)
➤ Higher speedup — one execution thread on all data sets	➤ Lower speedup — each processor runs different thread or process
➤ Parallelization proportional to input data size	➤ Parallelization proportional to number of independent tasks
➤ Designed for optimum load balance on multiprocessor systems	➤ Load balancing depends on hardware and scheduling algorithms
➤ All threads execute the same code on different data	➤ Each thread executes a different code or task
➤ Focus is on distributing data across processors	➤ Focus is on distributing tasks across processors
➤ Best for data processing apps like databases	➤ Best for multi-step apps like image processing
➤ Tasks are generally independent of each other	➤ Tasks may or may not depend on each other
➤ Example: Each CPU sorts a different portion of an array	➤ Example: One CPU compresses while another encrypts a file

Distributed Shared Memory (DSM):

Distributed Shared Memory (DSM) is a memory architecture where physically separated memories are combined and addressed as a single logically shared address space. Even though memory is spread across different machines, programmers can write code as if working with one unified memory.

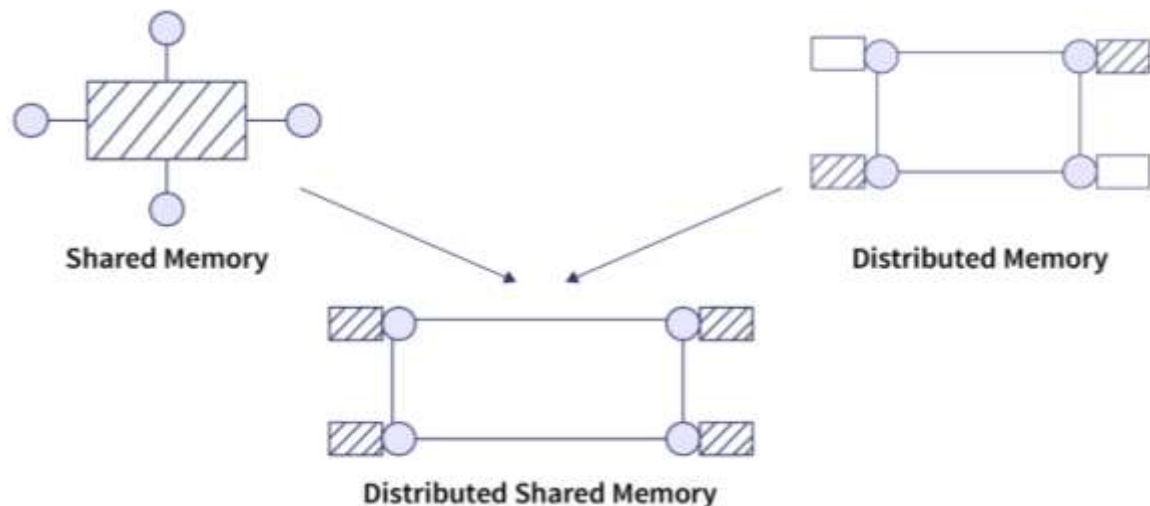
A DSM system consists of multiple independent nodes, each having its own local memory, all interconnected through a network. This combination gives the feel of shared memory while physically being distributed — making it a powerful middle ground between shared and distributed memory systems.

Advantages of DSM:

- Works efficiently with a **large number of nodes**
- Communication is handled **automatically** without manual message passing
- Can handle **large and complex databases** without needing to replicate or send data to processes
- Generally **cheaper** than building a full multiprocessor system
- Provides a **large virtual memory space** — more addressable memory than physical memory

Disadvantages of DSM:

- **Slower to access** than non-distributed shared memory due to network communication delays
- Requires **additional protection** to handle simultaneous access to shared data correctly
- May incur a **performance penalty** due to network communication overhead
- Offers **little programmer control** over the actual messages being generated internally



Distributed Shared Memory (DSM) Primitives:

DSM Primitives are the basic operations that a DSM system provides to programmers to access, manage and synchronize shared memory across multiple nodes. Some common DSM Primitives are:

Primitive	Symbol Example	What it does
Read	$R(x)$	Reads x data from a shared memory location
Write	$W(x)$	Write x data to a shared memory location
Lock	$L(x)$	Locks x memory region so only one process can access it
Unlock	$UL(x)$	Releases x memory so other processes can access it
Barrier	$B()$	Forces all processes to wait until everyone reaches the same point
Broadcast	$BC(x)$	Sends data from one node to all other nodes.

Unit 4

Parallel Algorithms I

Parallel Algorithms:

An algorithm is a step-by-step procedure that takes an input, performs some computation, and produces an output. A parallel algorithm works by breaking a large problem into smaller sub-problems, which are then solved simultaneously across multiple processing devices. Each device works on its own piece of the problem independently, and once all pieces are solved, the individual results are merged together to form the final output.

Monte Carlo Methods:

Monte Carlo methods are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. The core idea is to use randomness to solve problems that would otherwise be difficult or impossible to solve directly. These methods are particularly useful when a problem involves too many variables or uncertainties to compute in a straightforward way.

Monte Carlo methods generally follow these steps:

1. Determine the statistical properties of the possible inputs
2. Generate many sets of possible inputs that follow those properties
3. Perform a deterministic calculation on each of those input sets
4. Statistically analyze all the results to get the final answer

Example: Primality Testing Using Fermat's Little Theorem

Fermat's Little Theorem states that if p is a prime number and b is a positive integer not divisible by p , then:

$$b^{p-1} \equiv 1 \pmod{p}$$

If this condition holds, p is likely prime. If it does not, p is definitely composite.

Algorithm:

```
bool isPrime(int x) {  
    choose a random integer a, where  $1 < a < x$   
    result = power(a, x-1) mod x  
    if (result == 1)  
        return TRUE           // probably prime  
    else  
        return FALSE          // definitely composite  
}
```

Fermat's Little Theorem Example:

Let $p = 7$, $b = 3$

$$b^{p-1} = 3^{7-1} = 3^6 = 729$$

$$729 \bmod 7 = 1$$

Since $1 \equiv 1 \pmod{7} \checkmark \rightarrow 7$ is prime

Note: This is a probabilistic algorithm – It can confirm a number is composite with certainty, but can only say a number is probably prime.

Running it multiple times with different values of 'a' increases the confidence of number being prime.

Applications of Monte Carlo Methods:

Monte Carlo methods are widely used across many different fields due to their ability to handle complex, uncertain, and large-scale problems through random sampling. The key application areas are described below:

- ✚ **Radiation transport** — simulating how particles travel and interact through materials
- ✚ **Nuclear criticality** — determining conditions required to sustain a nuclear chain reaction
- ✚ **Design of nuclear weapons** — modeling and analyzing nuclear weapon behavior
- ✚ **Phase transitions** — studying how matter changes from one state to another (e.g., solid to liquid)
- ✚ **Atomic wave functions and eigenvalues** — computing quantum mechanical properties of atoms
- ✚ **Intra-nuclear cascade reactions** — simulating particle collisions inside a nucleus
- ✚ **Long chain coiling of polymers** — understanding how large molecules fold and behave
- ✚ **Partial differential equations** — approximating solutions to complex mathematical equations
- ✚ **Numerical integration** — estimating the value of integrals that are hard to solve analytically
- ✚ **Development of statistical tests** — designing and validating methods for statistical analysis
- ✚ **Combinatorial problems** — solving problems that involve counting or arranging elements
- ✚ **Signal detection** — identifying useful signals from noisy or uncertain data
- ✚ **Operations research** — optimizing complex decision-making processes in business and systems
- ✚ **Design of nuclear reactors** — simulating reactor behavior for safe and efficient design
- ✚ **Statistical physics** — analyzing physical systems with large numbers of particles
- ✚ **Wetting and growth of thin films** — modeling how films form and spread on surfaces
- ✚ **Thermodynamic properties** — estimating energy, entropy, and other thermal properties of substances
- ✚ **Reaction kinetics** — modeling how chemical reactions proceed over time
- ✚ **Large sets of linear equations** — solving massive equation systems that are computationally expensive
- ✚ **Uncertainty analysis** — quantifying how errors or variations in inputs affect the output
- ✚ **Cell population studies** — simulating how cells grow, divide, and interact over time
- ✚ **Search and optimization** — finding the best solution among a large number of possibilities
- ✚ **War games** — modeling and analyzing strategic military conflict scenarios

Random Numbers:

A random number is a value drawn from a sequence, where following two conditions must be satisfied.

1. The values must be uniformly distributed over a defined interval, meaning every value has an equal chance of occurring.
2. it must be impossible to predict any future value based on past or present values.

Together, these two properties ensure true randomness. Random numbers play a fundamental role in statistical analysis and probability theory, and are the backbone of methods like Monte Carlo simulations.

Parallel Random Number Generation (RNG):

Random Number Generation (RNG) is the process of producing a sequence of numbers or symbols that cannot be predicted in any reasonable way — essentially, each value appears by pure chance. There are two main types of random number generators:

1. Hardware Random Number Generators (HRNGs):

These are truly random. They generate numbers based on constantly changing physical properties of the environment, such as electrical noise, thermal activity, radio-active decay, etc. Since these physical processes are practically impossible to model or predict, the numbers produced are genuinely random.

2. Pseudo Random Number Generators (PRNGs):

These only appear to be random. In reality, the numbers are generated using a deterministic mathematical formula, meaning if you know the internal state of the generator, you can reproduce the exact same sequence of numbers. Despite not being truly random, PRNGs are widely used in practice because they are fast, efficient, and good enough for most computational purposes.

Some common Pseudo Random Number Generator methods are:

- a. Linear Congruential Method
- b. Mid Square Method, etc.

Practical Applications of Parallel Random Number Generation:

Random number generators are used in a wide variety of fields wherever unpredictable or unbiased results are needed. The key application areas are:

Gambling and Gaming

Random number generators ensure fair and unpredictable outcomes in games, lotteries, and casinos.

Statistical Sampling

Used to select random samples from large datasets for surveys, experiments, and analysis.

Computer Simulations

Particularly in Monte Carlo simulations, where the ability to rerun the same sequence of random numbers by using the same seed is extremely useful for testing and debugging.

Cryptography

Random numbers are used to generate secret keys for encrypting communication. Both the sender and receiver can independently generate the same set of numbers using a shared secret seed, which is then used as an encryption key.

Completely Randomized Design:

Used in scientific experiments to randomly assign subjects to groups, eliminating bias.

It is worth noting that not all applications require the same level of randomness. Cryptography and certain numerical algorithms demand a very high degree of unpredictability, while many other tasks such as simulations or games only need a modest level of randomness. This is why PRNGs, despite not being truly random, are sufficient and widely used across most of these applications.

General Approaches to Random Number Generation on Parallel Computers:

There are three general approaches to generating random numbers on parallel computers. They are:

1. Centralized Approach:

A single random number generator serves all tasks in the system. Whenever a task needs a random number, it sends a request to this central generator and waits for a response. This sounds simple, but it creates two serious problems. Since all tasks depend on one generator, it becomes a bottleneck and slows down the entire system. Also, because results depend on the exact timing of requests, the same program may give different results each time it runs, making it hard to reproduce results.

2. Replicated Approach:

Each task has its own random number generator instead of using a central one. These generators may start with the same or different seeds (like task ID). This method is fast and easy to use, but the random numbers from different tasks can sometimes be related (not fully independent), which may reduce accuracy in some applications.

3. Distributed Approach:

A single random number sequence is generated and then divided into portions, with each task receiving its own portion. Since all these generators are derived from one master generator, analyzing and verifying their statistical quality becomes straightforward. Among the three approaches, the distributed approach is considered the best as it provides both good performance and reliable statistical properties, making it the most suitable choice for parallel computing.

Random Number Distribution:

A random number distribution describes how random numbers are spread or arranged over a given range. It defines the probability of each possible value occurring. The most commonly used distributions are:

1. Uniform Distribution:

In this distribution, every value within a given range has an equal probability of occurring. For example, rolling a fair die gives each number from 1 to 6 an equal chance. This is the most basic and widely used distribution in random number generation.

2. Normal (Gaussian) Distribution:

In this distribution, values are distributed symmetrically around a mean (average), forming a bell-shaped curve. Most values cluster near the mean, and very high or very low values are rare. This distribution is extremely common in nature and statistics.

3. Exponential Distribution:

This distribution models the time between events in a process where events occur continuously and independently at a constant rate. Commonly used in queuing theory and reliability analysis.

4. Binomial Distribution:

This distribution describes the number of successes in a fixed number of independent trials, where each trial has only two possible outcomes — success or failure. For example, flipping a coin 10 times and counting the number of heads.

5. Poisson Distribution:

This distribution models the number of times an event occurs within a fixed interval of time or space. For example, the number of customers arriving at a shop per hour.

6. Bernoulli Distribution:

This distribution is the simplest distribution, where a random variable has only two possible outcomes – 1 (success) with probability p , and 0 (failure) with probability $1-p$.

Row-wise block-stripped Algorithm:

In this approach, a matrix is divided into groups of rows and distributed across multiple processors. Each processor is assigned a contiguous block of rows to work on independently – this is what makes it "block-stripped."

How the Matrix is Divided:

For an $n \times n$ matrix distributed across p processors ($P_0, P_1, P_2, \dots, P_{p-1}$), each processor receives exactly n/p rows. Specifically, processor P_i is assigned rows from index:

$$(n/p) \times i \text{ up to } (n/p) \times (i+1) - 1$$

For example, if we have a 16×16 matrix and 4 processors, each processor gets exactly 4 rows.

How the computation works:

Consider multiplying matrix A ($n \times n$) by vector x ($n \times 1$):

- The matrix A is divided row-wise among p processors, so each processor holds n/p rows of A .
- The vector x is made available to all processors.
- Each processor P_i computes the dot product of its assigned rows of A with the vector x , storing the partial results in its local array y .
- Finally, processor P_0 collects all the partial results from every processor and combines them to produce the final output vector y .

This approach is simple and efficient because each processor works independently on its own rows, minimizing the need for communication between processors until the very final step of collecting results.

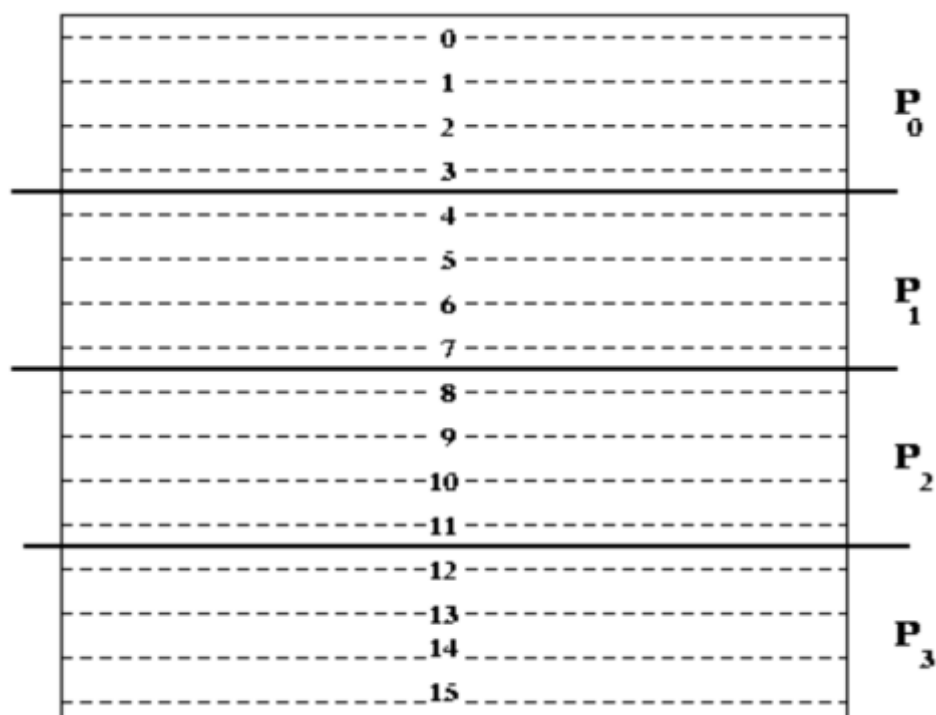


Fig: Row-wise block stripping

Cannon's Algorithm:

Cannon's algorithm is a distributed algorithm for matrix multiplication. It is designed specifically for systems where processors are arranged in a 2D mesh (grid) layout. The main advantage of this algorithm is that its memory requirements remain constant regardless of the number of processors used, making it very memory efficient.

Basic Idea:

Instead of having one processor handle the entire matrix multiplication, the matrices are broken into smaller blocks and distributed across processors arranged in a 2D grid. Each processor is responsible for computing one small block of the final result matrix.

Working:

Consider two $n \times n$ matrices A and B, and p processors arranged in a $\sqrt{p} \times \sqrt{p}$ grid. Each processor $P(i,j)$ is assigned one block from A and one block from B, each of size $(n/\sqrt{p}) \times (n/\sqrt{p})$, and is responsible for computing the corresponding block $C(i,j)$ of the final result matrix.

Algorithm:

1. Initial Alignment:

Before computation begins, the blocks are rearranged so that each processor has the correct pair of blocks to start multiplying. This is done by:

- Shifting each block $A(i,j)$ to the **left** by i steps
- Shifting each block $B(i,j)$ **upward** by j steps

2. Local Multiplication:

After alignment, each processor independently multiplies its local block of A with its local block of B, and stores the result as a partial sum in $C(i,j)$.

3. Shift and Repeat:

($\sqrt{p} - 1$ more times)

After each multiplication:

- All blocks of A shift **one step left**
- All blocks of B shift **one step up**

Each processor then multiplies its new pair of blocks and adds the result to its existing partial sum. This shifting and multiplying repeats until all blocks have been processed and the final result matrix C is complete.

Note:

Practice CANNON'S Algorithm
numericals by yourself !!



Cannon's Algorithm Explained YouTube link!!

The matrix elements are shifted as:

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$
$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$

(a) Initial alignment of A

$B_{0,0}$	$B_{0,1}$	$B_{0,2}$	$B_{0,3}$
$B_{1,0}$	$B_{1,1}$	$B_{1,2}$	$B_{1,3}$
$B_{2,0}$	$B_{2,1}$	$B_{2,2}$	$B_{2,3}$
$B_{3,0}$	$B_{3,1}$	$B_{3,2}$	$B_{3,3}$

(b) Initial alignment of B

We perform shift in A matrix by 'i' blocks, and in B matrix by 'j' blocks.

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$B_{0,0}$	$B_{1,1}$	$B_{2,2}$	$B_{3,3}$
$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	$A_{1,0}$
$B_{1,0}$	$B_{2,1}$	$B_{3,2}$	$B_{0,3}$
$A_{2,2}$	$A_{2,3}$	$A_{2,0}$	$A_{2,1}$
$B_{2,0}$	$B_{3,1}$	$B_{0,2}$	$B_{1,3}$
$A_{3,3}$	$A_{3,0}$	$A_{3,1}$	$A_{3,2}$
$B_{3,0}$	$B_{0,1}$	$B_{1,2}$	$B_{2,3}$

(c) A and B after initial alignment

$A_{0,1}$	$A_{0,2}$	$A_{0,3}$	$A_{0,0}$
$B_{1,0}$	$B_{2,1}$	$B_{3,2}$	$B_{0,3}$
$A_{1,2}$	$A_{1,3}$	$A_{1,0}$	$A_{1,1}$
$B_{2,0}$	$B_{3,1}$	$B_{0,2}$	$B_{1,3}$
$A_{2,3}$	$A_{2,0}$	$A_{2,1}$	$A_{2,2}$
$B_{3,0}$	$B_{0,1}$	$B_{1,2}$	$B_{2,3}$
$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$
$B_{0,0}$	$B_{1,1}$	$B_{2,2}$	$B_{3,3}$

(d) Submatrix locations after first shift

We make separate matrix C to hold the partial result of final result matrix. Which is not shown here. But when we solve numericals, we need to make it.

$A_{0,2}$	$A_{0,3}$	$A_{0,0}$	$A_{0,1}$
$B_{2,0}$	$B_{3,1}$	$B_{0,2}$	$B_{1,3}$
$A_{1,3}$	$A_{1,0}$	$A_{1,1}$	$A_{1,2}$
$B_{3,0}$	$B_{0,1}$	$B_{1,2}$	$B_{2,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$
$B_{0,0}$	$B_{1,1}$	$B_{2,2}$	$B_{3,3}$
$A_{3,1}$	$A_{3,2}$	$A_{3,3}$	$A_{3,0}$
$B_{1,0}$	$B_{2,1}$	$B_{3,2}$	$B_{0,3}$

(e) Submatrix locations after second shift

$A_{0,3}$	$A_{0,0}$	$A_{0,1}$	$A_{0,2}$
$B_{3,0}$	$B_{0,1}$	$B_{1,2}$	$B_{2,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$B_{0,0}$	$B_{1,1}$	$B_{2,2}$	$B_{3,3}$
$A_{2,1}$	$A_{2,2}$	$A_{2,3}$	$A_{2,0}$
$B_{1,0}$	$B_{2,1}$	$B_{3,2}$	$B_{0,3}$
$A_{3,2}$	$A_{3,3}$	$A_{3,0}$	$A_{3,1}$
$B_{2,0}$	$B_{3,1}$	$B_{0,2}$	$B_{1,3}$

(f) Submatrix locations after third shift

Solving Linear Systems:

A system of linear equations is a collection of two or more equations with the same set of variables. Consider the following example:

$$\begin{aligned}x + y &= 5 \\ 2x - y &= 1\end{aligned}$$

We have following methods to solve system of linear methods. They are:

1. Direct Methods
 - a. Gauss Elimination with back substitution
2. Indirect Methods or Iterative Methods
 - a. Gauss jacobi Method
 - b. Gauss Seidal Method



Gauss Elimination with Back Substitution:

Gaussian elimination is a method for solving a system of linear equations by transforming it into a simpler form that is easy to solve. It gives exact solutions. It works in two phases:

1. Phase 1 [Gaussian Elimination]:

In this phase, the system of equations is transformed into an upper triangular form. This means all the elements below the main diagonal are made zero by performing row operations. This process is repeated for each column, moving from left to right, until the matrix looks like a staircase shape with zeros below the diagonal.

2. Phase 2 [Gaussian Substitution]:

Once the system is in upper triangular form, back substitution is applied. The last equation now contains only one unknown, so it is solved directly. That value is then substituted back into the second to last equation to find the next unknown, and so on, moving upward through the equations one by one until all unknowns are found.

Gauss Jacobi Method:

The Gauss-Jacobi method is an iterative method for solving a system of linear equations. It starts with an initial guess for all unknowns, usually zero, and keeps refining them repeatedly until the solution converges. In each iteration, every unknown is computed using the values of all other unknowns from the previous iteration only, and all unknowns are updated simultaneously at the end of each iteration.

This process repeats until the difference between two consecutive iterations falls below a defined threshold value called tolerance, at which point the solution is considered accurate enough. Since all unknowns are computed independently using only the previous iteration's values, Gauss-Jacobi is simple to parallelize, making it well suited for parallel computing.

Gauss Seidal Method:

The Gauss-Seidel method is an iterative method for solving a system of linear equations, similar to the Gauss-Jacobi method. It also starts with an initial guess for all unknowns, usually zero, and keeps refining them repeatedly. However, the key difference is that in Gauss-Seidel, each unknown is updated immediately as soon as it is computed, meaning the most recently updated values are used within the same iteration rather than waiting for the iteration to complete.

This makes Gauss-Seidel converge faster than Gauss-Jacobi. The method stops when the difference between two consecutive iterations falls below a defined tolerance value, at which point the solution is considered accurate enough. However, because each unknown depends on the most recently computed values of other unknowns, it is harder to parallelize compared to Gauss-Jacobi.



Gauss Seidal method



Gauss Jacobi Method

Conjugate Gradient Method:

The Conjugate Gradient method is an iterative method used to solve large and sparse systems of linear equations, where most values in the matrix are zero. Unlike Gauss-Jacobi and Gauss-Seidel, it is smarter in the way it converges – in each iteration it chooses the best possible direction to move toward the solution, ensuring maximum progress without undoing previous steps. This property is called conjugacy.

The method starts with an initial guess, computes the residual – the difference between the current approximation and the true solution – and uses it to determine the next best direction to move in. This repeats until the residual becomes small enough and the solution converges. It is especially powerful for parallel computing because its core operations such as matrix-vector multiplication and dot products can be efficiently parallelized.

Unit 5

Parallel Algorithm II

Quicksort:

Quicksort is a sorting algorithm based on the divide and conquer approach. The array is split into subarrays and these subarrays are recursively sorted.

Parallel Quicksort:

Parallel quicksort is a way to speed up quicksort by distributing the sorting task to multiple processors.

Working of parallel quicksort:

A pivot is selected by one processor and broadcast to all processors. Each processor then splits its local data into two parts – values smaller than pivot are on left and values greater than pivot are on right – and exchanges the appropriate part with its partner processors. This process continues for $\log P$ steps [P is no. of processors]. Finally, each process sorts its own chunk using sequential quicksort to produce the fully sorted array.

Algorithm for Parallel Quicksort:

1. **Distribute Data**

Divide the entire data equally among all **P processors** ($P_0, P_1, P_2 \dots P_n$).

2. **Choose Pivot**

One processor selects a pivot and broadcasts it to all processors. **Note:** The pivot element should be chosen by sampling elements from the local data and it should be nearest to the median so that the data is split as evenly as possible between 2 halves.

3. **Local Split**

Each processor splits its local data into two parts:

- Elements $\leq$ **pivot** $\rightarrow$ left side (Low list)
- Elements $>$ **pivot** $\rightarrow$ right side (High list)

4. **Exchange**

- **Lower half processors** ($P_0, P_1 \dots$) $\rightarrow$ keep low list, send high list to upper half
- **Upper half processors** ($P_2, P_3 \dots$) $\rightarrow$ keep high list, receive low list from lower half

5. **Repeat**

Each half group repeats **Steps 2, 3, 4** recursively within themselves.

6. **After $\log P$ Steps**

Each processor holds values that are **unique to it and not shared with any other processor**.

7. **Local Sort**

Each processor sorts its own chunk using **sequential quicksort**.

Example: Sorting an Array using Parallel Quicksort

$A = \{7, 13, 18, 2, 17, 1, 14, 20, 6, 10, 15, 9, 3, 16, 19, 4, 11, 12, 5, 8\}$

Soln:

First Step

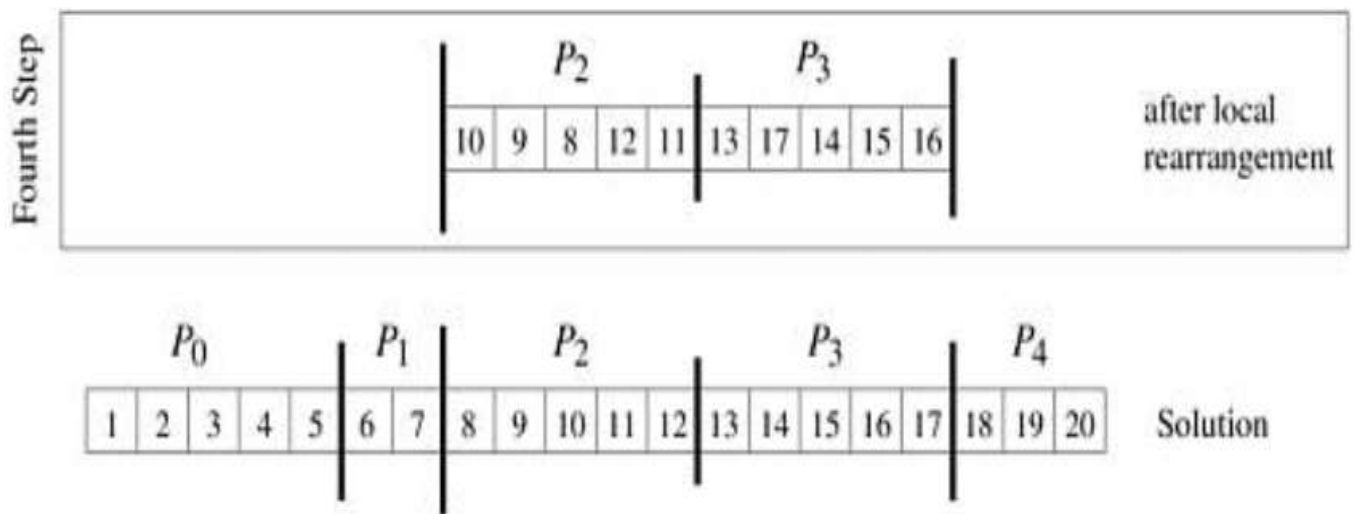
P_0	P_1	P_2	P_3	P_4																
7	13	18	2	17	1	14	20	6	10	15	9	3	16	19	4	11	12	5	8	pivot selection
pivot=7																				
P_0	P_1	P_2	P_3	P_4																
7	2	18	13	1	17	14	20	6	10	15	9	3	4	19	16	5	12	11	8	after local rearrangement
7	2	1	6	3	4	5	18	13	17	14	20	10	15	9	19	16	12	11	8	after global rearrangement

Second Step

P_0	P_1		P_2	P_3	P_4														
7	2	1	6	3	4	5	18	13	17	14	20	10	15	9	19	16	12	11	8
pivot=5							pivot=17												
pivot selection																			
P_0	P_1		P_2	P_3	P_4														
1	2	7	6	3	4	5	14	13	17	18	20	10	15	9	19	16	12	11	8
after local rearrangement																			
P_0	P_1		P_2	P_3	P_4														
1	2	3	4	5	7	6	14	13	17	10	15	9	16	12	11	8	18	20	19
after global rearrangement																			

Third Step

P_0	P_1	P_2	P_3	P_4																	
1	2	3	4	5	7	6	14	13	17	10	15	9	16	12	11	8	18	20	19	pivot selection	
pivot=11																					
P_0	P_1	P_2	P_3	P_4																	
1	2	3	4	5	6	7	10	13	17	14	15	9	8	12	11	16	18	19	20	after local rearrangement	
							10	9	8	12	11	13	17	14	15	16					after global rearrangement



Hypercube:

(or, Hypercube connection)

A hypercube is a network topology used in parallel computing where processors are connected to each other in a cube-like structure. Each node in the hypercube represents a processor, and edges between nodes represent communication links. The features of hypercube are:

- A hypercube of dimension d has 2^d nodes, where each node is connected to d other nodes.
- It can easily scale by increasing dimension d , allowing more processors to be added.
- Nodes communicate directly with their neighbors enabling efficient and fast data transfer.
- If a node fails, the remaining nodes can still communicate without interruption.
- It supports high degree of parallelism, making it suitable for a wide range of parallel algorithms.
- Addresses of directly connected processors differ exactly by 1 bit.

Prefix computation on Hypercube:

Prefix computation is a common operation in parallel computing where each processor computes a prefix sum (or any associative operation) over a distributed set of values. In a hypercube network, this operation is performed efficiently in $\log_2(N)$ steps using two phases:

1. Forward phase
2. Reverse phase

Forward Phase:

In the forward phase of prefix computation on a hypercube, each processor initially starts with its own value. The processors then communicate with their hypercube neighbors in a structured manner, where at each step, nodes differing in one bit position exchange their values.

After receiving data, each processor combines (aggregates) the received value with its own value to form partial prefix results. This process continues dimension by dimension (1-bit difference, 2-bit difference, and so on) for $\log_2(N)$ steps, gradually propagating and combining information across the hypercube.

By the end of the forward phase, each processor holds a partial sum that contributes to the final prefix computation.

Reverse Phase:

In the reverse phase, the partial results computed during the forward phase are propagated back through the hypercube network to finalize the prefix values. Each processor again communicates with its neighbors, but instead of simply combining values, it adjusts its stored value based on the received information to extract the correct prefix contribution.

This backward communication ensures that the aggregated results are properly distributed to all processors. After $\log_2(N)$ steps, every processor obtains its final correct prefix sum, completing the prefix computation on the hypercube.

Hyper Quicksort:

Hyper Quicksort is a parallel sorting algorithm designed to efficiently sort large datasets using a hypercube network topology. It is an extension of quicksort optimized for parallel computing environment.

Algorithm:

1. Divide unsorted data among all nodes.
2. Each processor sorts the data using sequential quicksort.
3. A pivot (usually the median) is chosen and is broadcasted to other nodes.
4. Each node splits its data into 2 parts ($\leq$ pivot on left side and $>$ pivot on right side) and exchanges the halves with its partner nodes.
5. Step 2 and 3 are repeated while reducing hypercube dimension each time, until the dimension becomes 0.
6. Each node then holds its final sorted chunk of data.

Fast Fourier Transform (FFT):

FFT is an algorithm used to quickly compute the Discrete Fourier Transform (DFT) of a sequence. It reduces the computation time and makes signal analysis much faster. DFT is a mathematical technique that converts a signal from the time domain into the frequency domain. In simple words, it breaks a signal into its sine and cosine components, showing which frequencies are present in it.

Relationship between FFT and DFT:

FFT is not a different transform – it is just a fast computational method for calculating the DFT.

So:

-  **DFT** = mathematical concept
-  **FFT** = efficient algorithm to compute DFT

Applications of FFT:

-  **Signal Processing:** Used to read and analyze sound waves and other types of signals.
-  **Image Processing:** Widely used in image processing technologies to analyze and filter images.
-  **Sound Engineering:** Helps in analyzing audio frequencies and sound quality.
-  **Seismology:** Used to analyze earthquake waves and ground vibrations.
-  **Voltage Measurements:** Helps in analyzing electrical signals and frequency activity.
-  **Equation Solving:** Used to solve various types of mathematical and engineering equations.

Combinatorial Search:

Combinatorial search is a technique in computer science and artificial intelligence used to solve problems by exploring a large number of possible solutions (search space). These problems are often complex and difficult to solve directly.

The main idea is to find a solution effectively by reducing unnecessary exploration the search space. It works by exploring different possible solutions in a structured way while using smart techniques or heuristics to make the search faster. Some algorithms guarantee the optimal solution, while others return the best solutions found within the exploration space.

Some combinatorial search algorithms are:

1. A* Search Algorithm
2. Branch and Bound
3. Minimax
4. Alpha-Beta pruning

A* Search Algorithm:

A* (A-star) is a graph traversal and pathfinding algorithm widely used in artificial intelligence and computer science. It searches for the shortest and best path by combining the actual cost traveled with an estimated cost to the goal. It is highly popular because it is:

-  **Complete:** Always finds a solution if one exists.
-  **Optimal:** Always finds the best solution
-  **Efficient:** Finds it in the fastest way possible

Branch and Bound:

Branch and Bound is a search and optimization technique used to solve complex combinatorial problems. It divides a large problem into smaller subproblems (branching) and eliminates solutions that cannot produce a better result than the current best solution (bounding), reducing unnecessary computations.

Minimax:

Minimax is a decision-making algorithm used in two-player games. It analyzes possible future moves by assuming one player tries to maximize the score while the opponent tries to minimize it. The algorithm then selects the move that gives the best possible outcome against the opponent.

Alpha-Beta Pruning:

Alpha-Beta Pruning is an improved version of the Minimax algorithm that reduces unnecessary searching in a game tree. It skips evaluating moves that cannot affect the final decision, making the search faster and more efficient. It is commonly used in games like Chess, Tic-tac-toe, and Go.

In **alpha-beta pruning**, we use two parameters:

-  **Alpha (α):**
It represents the best (maximum) value that the MAX player can guarantee so far. It represents the highest lower bound of possible scores.
-  **Beta (β):**
It represents the best (minimum) value that the MIN player can guarantee so far. It represents the lowest upper bound of possible scores.

A cutoff happens when we stop exploring a branch because it cannot affect the final decision. **An alpha cutoff** occurs when the current value at a MIN node becomes less than or equal to alpha (α), meaning MAX already has a better option, so this branch is useless.

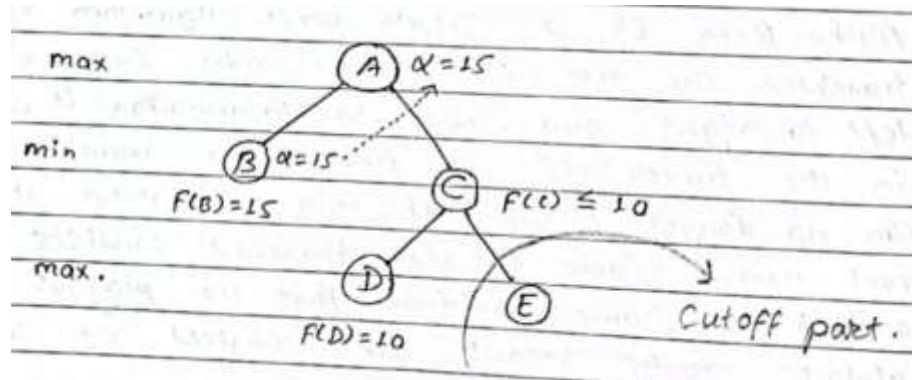


Fig: Alpha cutoff

A beta cutoff occurs when the current value at a MAX node becomes greater than or equal to beta (β), meaning MIN already has a better (smaller) option, so further exploration is unnecessary. In both cases, the condition $\alpha \geq \beta$ leads to pruning of the remaining branches.

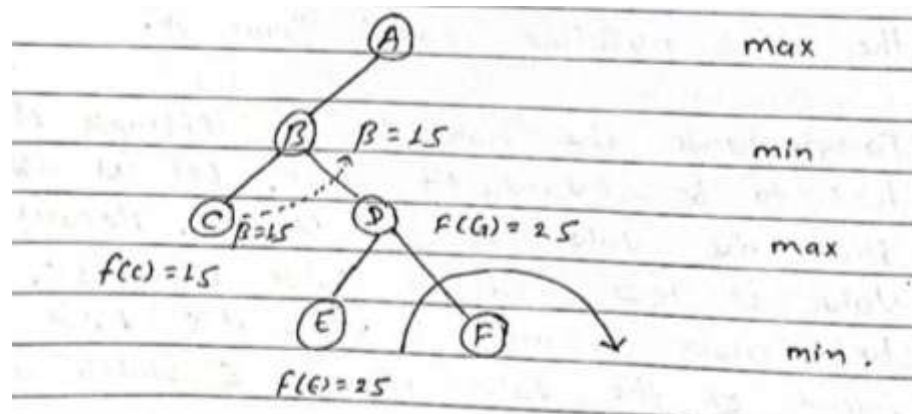


Fig: Beta cutoff

Branch and Bound:

Branch and Bound is an algorithm used to solve optimization and combinatorial problems by systematically searching through all possible solutions while eliminating unnecessary ones along the way. **Example: Travelling Salesman Problem (TSP).**

Working:

It has 2 main operations. They are:

- ✚ **Branching** → Splits the problem into smaller subproblems like a tree structure, exploring each possible solution one by one.
- ✚ **Bounding** → For each subproblem, it calculates a bound (best possible value) that can be achieved. If the bound is worse than the best solution already found, that subproblem is discarded (pruned).

Example of combinatorial Search:

[IMP QN + Numerical]

Travelling salesman Problem (TSP):

The Travelling Salesman Problem is a classic optimization problem where a salesman must visit all cities exactly once and return to the starting city with the minimum possible total distance.

As the number of cities increases, the number of possible routes increases drastically. I.e.

Cities	Possible Routes
4 cities	3 routes
5 cities	12 routes
10 cities	181,440 routes
20 cities	60,822,550,204,416,000 routes

Yo table khali dekhauna lai matra ho, yaad garnu pardaina... Its just for showing how the number of possible routes increases with increase in no. of city.

We cannot check all these routes, so we use the Branch and Bound algorithm to avoid evaluating every possible route.

Must Watch this video for TSP numerical using bound Branch and Bound algorithm



Parallel Alpha Beta Search:

Parallel Alpha-Beta Search is simply Alpha-Beta Pruning running on multiple processors simultaneously to evaluate more nodes faster and make better decisions in less time.

Why parallel?

In regular Alpha-Beta Search, nodes are evaluated one at a time on a single processor. For complex games like Chess, the search tree is enormous and takes too long. So, we distribute the work across multiple processors.

How it works?

1. The search tree is divided among multiple processors
2. Each processor evaluates its own portion of the tree simultaneously
3. Processors share alpha and beta values with each other
4. If one processor finds a better move, it updates the shared values so other processors can prune their branches accordingly

Divide and Conquer Algorithm:

Divide and Conquer is an algorithm design approach used in computer science to solve complex problems efficiently.

In this approach, a large problem is repeatedly divided into smaller subproblems of the same or related type until they become simple enough to solve directly. The solutions of these smaller subproblems are then combined to obtain the final solution to the original problem.

Divide and Conquer algorithms are well suited for parallel and multi-processor systems because different subproblems can be solved independently on different processors at the same time.

The steps of Divide and Conquer algorithm are:

1. **Divide:** Break the original problem into smaller subproblems.
2. **Conquer:** Solve each subproblem individually, usually using recursion.
3. **Combine:** Merge the solutions of the subproblems to get the final solution to the original problem.

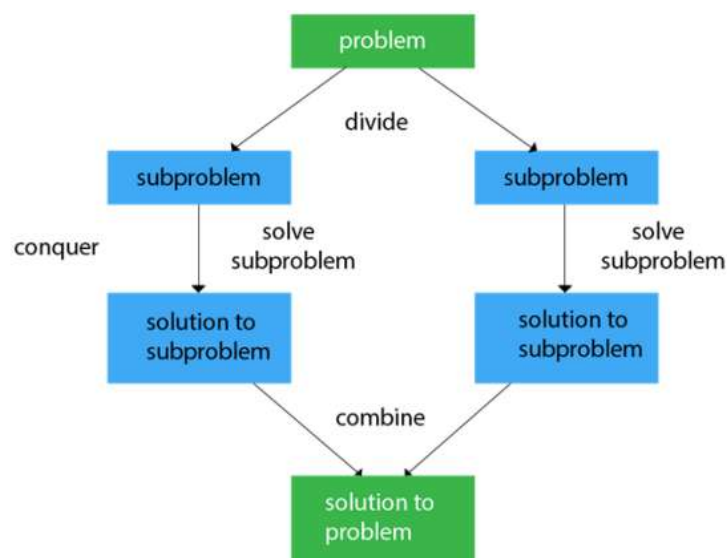


Fig: Divide and Conquer algorithm diagrammatic representation

Following algorithms use divide and conquer approach:

1. Binary search
2. Sorting algorithm (Merge sort, Quick sort)
3. Tower of Hanoi, etc.

Parallel Backtrack search:

Backtracking is a general problem-solving technique used to find all possible solutions or a valid solution for computational problems, especially constraint satisfaction problems.

It works by building a solution step by step. If at any stage the current path cannot lead to a valid solution, the algorithm stops exploring that path and goes back (backtracks) to try another possibility.

Algorithm steps:

1. Check if the current position is the goal state.
 - a. If yes, return success.
2. Otherwise, check whether the current position is a dead end.
 - a. If yes, return failure and backtrack.
3. If the current position is not a dead end, explore the next possible choices recursively.
4. Repeat the process until a solution is found or all possibilities are explored.

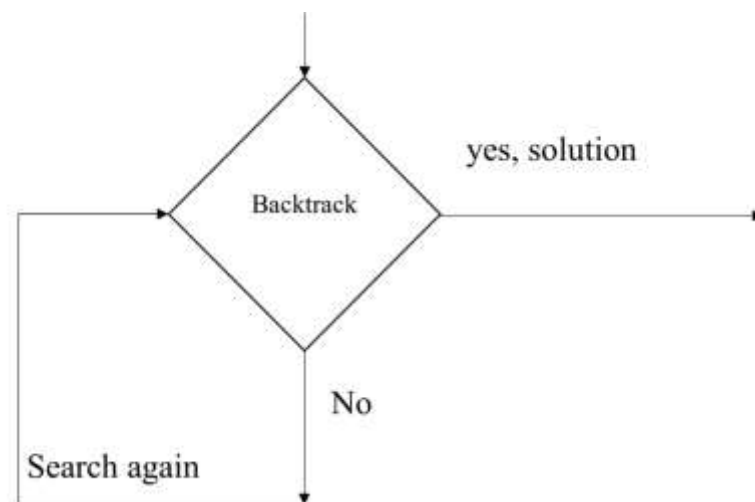


Fig: Diagrammatic representation of Backtracking search

Missing Exam Question

1. Explain machine model and CU/PU overlap.

Machine Model:

A machine model in parallel computing describes how a parallel system is structured and how computation and communication happen inside it.

-  **Processing Units (PU):** Multiple processors or cores that execute instructions
-  **Control Unit (CU):** Coordinates and controls execution of instructions
-  **Memory system:** Stores data and instructions (can be shared or distributed)
-  **Communication network:** Connects processors to exchange data

Common machine models include Shared memory model, Distributed memory model, Hybrid model, PRAM model, SIMD model, MIMD model, etc.

CU/PU Overlap:

CU/PU overlap refers to overlapping the work of the Control Unit (CU) and Processing Unit (PU) to improve performance. In simple words, CU works on 'next instruction' while PU works on 'current instruction'.

In a normal system:

1. CU fetches and decodes instructions
2. PU executes them
3. These happen one after another (no overlap)

In CU/PU overlap:

1. CU continues fetching/decoding next instructions
2. While PU is executing current instructions
3. This reduces idle time and increases speed

The benefits of CU/PU overlap are:

-  Improves CPU utilization
-  Reduces execution time
-  Increases instruction throughput

4. How can you solve a linear System?

A linear system is a set of linear equations with multiple unknowns. It can be solved using different methods such as:

1. Gaussian Elimination method
2. Gauss-Jordan Elimination
3. Gauss Jacobi Method
4. Gauss Seidal Method
5. Conjugate Gradient Method, etc.

5. Explain clustering.

Clustering is a technique used to group similar data items together based on their characteristics without using any predefined labels. In this method, items within the same group (cluster) are more similar to each other, while items in different groups are significantly different.

Since no training data or labels are provided, clustering is considered an unsupervised learning technique, where the algorithm itself discovers patterns in the data and forms meaningful groups. Some common types of clustering include hierarchical clustering, partition clustering, and density-based clustering.

6. Describe speedup gained due to pipelining.

Speedup gained due to pipelining refers to the improvement in performance achieved when multiple instruction stages are overlapped instead of executing instructions one after another.

In a non-pipelined system, each instruction must complete all stages sequentially, while in a pipelined system, different stages (such as fetch, decode, and execute) operate simultaneously on different instructions, which increases overall throughput.

Speedup:

The speedup is defined as the ratio of execution time without pipelining to execution time with pipelining.

If a pipeline has k stages and n instructions, then non-pipelined execution takes $n \times k$ cycles, while pipelined execution takes approximately $(n + k - 1)$ cycles.

$$\therefore \text{speedup} = (n \times k) / (n + k - 1)$$

In an ideal case, when the number of instructions is large, the speedup approaches k , meaning the performance improvement becomes nearly equal to the number of pipeline stages.

7. Differentiate between one-to-all broadcast and all-one reduction.

One-to-All Broadcast	All-to-One Reduction
➤ A single process sends data to all other processes	➤ All processes send data to one process
➤ Data is distributed from one source	➤ Data is collected from all sources
➤ Used to share information with all processes	➤ Used to combine results from all processes
➤ One sender, many receivers	➤ Many senders, one receiver
➤ No data combination is done	➤ Data is combined using operations like sum, max, min, etc.

8. Differentiate between parallel and serial computing.

Parallel Computing	Serial Computing
➤ Multiple tasks are executed simultaneously	➤ One task is executed at a time
➤ Uses multiple processors or cores	➤ Uses a single processor
➤ Faster execution for large problems	➤ Slower for large problems
➤ Problem is divided into smaller parts	➤ Problem is solved step by step
➤ Higher efficiency and performance	➤ Lower efficiency for heavy workloads
➤ More complex to design and implement	➤ Simple to design and implement

9. Explain NUMA Memory architecture?

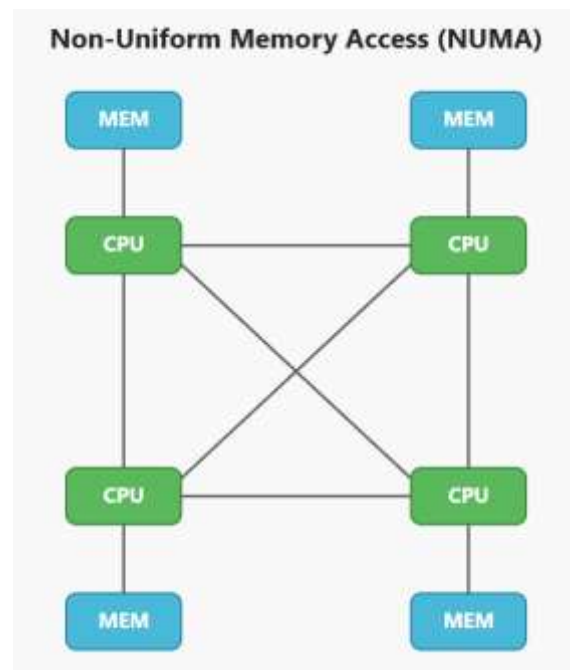
NUMA (Non-Uniform Memory Access) is a memory architecture used in multiprocessor systems where memory access time depends on the location of the memory relative to the processor.

In NUMA, each processor has its own local memory, but it can also access memory of other processors (called remote memory). The key point is that local memory access is faster than remote memory access, so memory access time is not uniform.

Each processor + its local memory is called a node, and all nodes are connected through an interconnection network. This allows processors to share a global memory space, but with different access speeds depending on distance.

This architecture is widely used in modern multiprocessor systems because it improves scalability and performance compared to shared memory systems. It reduces memory bottlenecks since processors mainly use their local memory, reducing load on a single shared memory.

However, performance in NUMA systems depends on data placement and memory management, as poor placement can lead to frequent remote memory access and reduced performance, so operating systems and compilers try to keep data close to the processor that uses it most.



10. What is PRAM Model? What are the subclasses of PRAM?

The PRAM (Parallel Random Access Machine) is a theoretical model used in parallel computing to design and analyze parallel algorithms. It assumes multiple processors working simultaneously with a shared memory, where all processors can access memory in synchronized steps.

The model is idealized, meaning it ignores practical issues like communication delay and focuses mainly on how parallel operations can be performed efficiently on shared memory.

The subclasses of PRAM are:

1. EREW PRAM (Exclusive Read Exclusive Write):

In this model, no two processors are allowed to read or write the same memory location at the same time. It is the most strict and safest form of PRAM.

2. **CREW PRAM (Concurrent Read Exclusive Write):**

In this model, multiple processors can read the same memory location simultaneously, but only one processor can write to a memory location at a time.

3. **ERCW PRAM (Exclusive Read Concurrent Write):**

In this model, only one processor can read a memory location at a time, but multiple processors are allowed to write to the same memory location simultaneously.

4. **CRCW PRAM (Concurrent Read Concurrent Write):**

In this most flexible model, multiple processors can read and write to the same memory location at the same time, but special rules are used to resolve write conflicts.

11. What is parallel reduction operation? Explain its types.

A parallel reduction operation is a technique in parallel computing used to combine data from multiple processors into a single result using an associative operation. Instead of processing all data sequentially, different processors perform computations simultaneously and then combine their partial results. This improves performance and reduces computation time in parallel systems.

The types of Parallel Reduction Operations are:

- ❖ **Sum Reduction:** Combines values from all processors by adding them together to produce a total sum.
- ❖ **Product Reduction:** Multiplies values from all processors to obtain a final product.
- ❖ **Maximum Reduction:** Finds the largest value among all processor data.
- ❖ **Minimum Reduction:** Finds the smallest value among all processor data.
- ❖ **Logical Reduction:** Applies logical operations such as AND, OR, or XOR on data from multiple processors.
- ❖ **User-defined Reduction:** Uses custom operations defined by the programmer to combine processor results according to application needs.

12. Explain the features of Parallel Object-Oriented Programming (POOP).

Parallel Object-Oriented Programming (POOP) combines the concepts of object-oriented programming with parallel computing to improve modularity, scalability, and performance in parallel applications. Its main features are:

- ❖ **Object-based decomposition:** Programs are divided into objects, where each object contains its own data and methods.
- ❖ **Parallel execution:** Multiple objects or methods can be executed simultaneously on different processors.
- ❖ **Encapsulation:** Data and functions are bundled together inside objects, improving data security and modularity.
- ❖ **Message passing between objects:** Objects communicate with each other by sending messages, which supports distributed execution.
- ❖ **Inheritance and polymorphism:** Object-oriented features like inheritance and polymorphism are supported, improving code reuse and flexibility.
- ❖ **Scalability:** Applications can easily expand by adding more objects or processors.
- ❖ **Synchronization and coordination:** Mechanisms are provided to coordinate parallel activities and avoid conflicts between objects.

- ❖ **Modularity:** Programs are easier to design, maintain, and debug because they are organized into independent objects.

13. What are parallel reduction algorithms? How are they used in PRAM architecture?

Parallel reduction algorithms are algorithms used in parallel computing to combine multiple data values into a single result using operations such as sum, product, maximum, minimum, or logical operations. Instead of processing data sequentially, the computation is divided among multiple processors, where each processor performs part of the operation simultaneously and partial results are combined step by step. This reduces computation time and improves performance for large datasets.

In PRAM (Parallel Random Access Machine) architecture, parallel reduction algorithms are widely used because multiple processors can access shared memory simultaneously. The reduction process is usually performed in a tree-like manner, where pairs of processors combine their results at each step until only one final result remains. For example, in sum reduction, processors add pairs of values in parallel, reducing the number of active values by half in every step. This allows PRAM systems to solve reduction problems efficiently in logarithmic time using multiple processors.

14. Discuss different parallel approaches including data parallelism and control parallelism.

Parallel computing uses different approaches to execute multiple operations simultaneously for improving performance and reducing execution time. The main parallel approaches are data parallelism and control parallelism.

- ❖ **Data Parallelism:**

In this approach, the same operation is performed simultaneously on different parts of a large dataset using multiple processors. The data is divided into smaller chunks, and each processor works on its assigned portion independently. It is commonly used in scientific computing, image processing, and matrix operations.

- ❖ **Control Parallelism (Task Parallelism):**

In this approach, different processors execute different tasks or instructions at the same time. Each processor may perform separate operations on the same or different data. It is useful when a problem can be divided into independent tasks running concurrently.

- ❖ **Pipeline Parallelism:**

The computation process is divided into multiple stages, and different stages are executed simultaneously by different processors, similar to an assembly line. Each stage processes part of the task and passes the result to the next stage.

- ❖ **Hybrid Parallelism:**

This approach combines multiple parallel techniques, such as data parallelism and task parallelism, to achieve better performance and scalability in complex applications.

15. How will you define speed up if a number of processors is added to perform fraction of work in parallel?

When a fraction of a program is executed in parallel using multiple processors, the speedup is defined as the ratio of the execution time using one processor to the execution time using multiple processors.

According to Amdahl's law, if:

$S = 1 - P$ = Fraction of work that must be executed serially (Non parallelizable portion)

P = Fraction of Work that can be executed in parallel

N = Number of processors.

S(N) = Speedup

Then, the speedup is given by:

$$S(N) = \frac{1}{s + \frac{p}{N}}$$

This means the serial part still takes the same time, while the parallel part is divided among N processors. As more processors are added, the execution becomes faster, but the serial portion limits the maximum possible speedup.

16. Explain routing?

Routing is the process of selecting a path for transferring data from a source node (sender) to a destination node (receiver) in a computer or parallel network.

In parallel computing systems, many processors are connected through an interconnection network. When one processor wants to send data to another processor, routing decides through which links and intermediate nodes the message should travel to reach its destination correctly and efficiently.

The main goal of routing is to:

- ❖ Deliver messages correctly
- ❖ Reduce communication delay
- ❖ Avoid congestion and deadlock
- ❖ Improve network performance

17. Explain Classification of Routing mechanism?

Routing mechanisms in parallel and interconnection networks are mainly classified into two types. They are:

1. Deterministic Routing
2. Adaptive Routing

Deterministic Routing:

In deterministic routing, the path between a source and destination is fixed in advance. For a given source and destination pair, the same route is always followed. Example: XY routing in mesh network.

The features are:

- ❖ Simple and easy to implement
- ❖ Predictable behavior
- ❖ Low overhead
- ❖ Cannot avoid congestion or faults easily

Adaptive Routing:

In adaptive routing, the path is selected dynamically depending on network conditions such as traffic, congestion, or faulty links. Example: Dynamic routing is used in multicomputer network.

The features are:

- ❖ Flexible routing
- ❖ Can avoid congested or failed paths
- ❖ Better performance under heavy traffic
- ❖ More complex than deterministic routing

18. Differentiate between deterministic routing and adaptive routing.

Deterministic Routing	Adaptive Routing
➤ The path between source and destination is fixed for a message	➤ The path is selected dynamically based on network conditions
➤ Same source and destination always use the same route	➤ Different routes may be used for the same source and destination
➤ Simple and easy to implement	➤ More complex to implement
➤ Does not consider traffic or congestion	➤ Considers traffic, congestion, or faults while selecting routes
➤ Less flexible	➤ More flexible and efficient
➤ May lead to network congestion	➤ Helps reduce congestion by choosing alternate paths
➤ Provides predictable behavior	➤ Provides better fault tolerance and load balancing

19. Why do we use MPT?

Message Passing Technique (MPT) is used in parallel computing systems where multiple processors need to work together but do not share memory. Instead, they communicate by sending messages. We use MPT because:

- ❖ Processors in distributed systems have **separate memory**, so they cannot directly access each other's data.
- ❖ It provides a way to **exchange data between processors**.
- ❖ It helps in **coordinating parallel tasks**.
- ❖ It supports **scalability**, meaning we can add more processors easily.
- ❖ It improves performance in **distributed and cluster computing systems**.

20. Explain different paradigms of parallel computing. Also discuss merits and demerits of parallel computation.

Parallel computing paradigms describe the different ways in which tasks are divided and executed simultaneously on multiple processors. The different paradigms are Data Parallelism, Task Parallelism, Pipeline Parallelism, and Functional parallelism.

Data Parallelism:

In data parallelism, the same operation is performed on different parts of data at the same time using multiple processors. The data is divided among processors, but each processor executes the same instruction. It is mainly used when large datasets need to be processed quickly, such as array operations.

Task Parallelism:

In task parallelism, different tasks or functions are executed simultaneously on different processors. Each processor may perform a different operation on the same or different data. It is useful when a program has independent tasks that can run at the same time.

Pipeline Parallelism:

In pipeline parallelism, a task is divided into several stages, and each stage is executed in parallel. As one data item moves through stages, different items are processed at different stages at the same time, similar to an assembly line. It improves throughput of the system.

Functional Parallelism:

In functional parallelism, different functions of a program run in parallel on different processors. Each function performs a separate part of the overall computation. It is useful when a program has multiple independent functions that can work simultaneously.

Merits of Parallel Computing:

- ❖ Faster execution (reduces computation time)
- ❖ Handles large problems efficiently
- ❖ Better resource utilization
- ❖ Cost-effective compared to a single powerful processor
- ❖ Scalable (can add more processors to improve performance)
- ❖ Solves complex problems that are not possible sequentially

Demerits of Parallel Computing:

- ❖ Complex programming and design
- ❖ High communication overhead between processors
- ❖ Synchronization issues
- ❖ Load balancing problems (some processors may be idle)
- ❖ Debugging is difficult
- ❖ Limited speedup due to serial part of program (Amdahl's Law)

21. Compare Amdahl's law and Gustafson Barsis Law.

Amdahl's Law	Gustafson–Barsis Law
➤ Assumes fixed size of the problem (same workload even if processors increase).	➤ Assumes problem size increases when more processors are added.
➤ Calculates speedup based on constant workload division.	➤ Calculates speedup based on scaled workload across processors.
➤ Shows that performance improvement is limited by the serial part of the program.	➤ Shows that performance improves because parallel part increases with workload.
➤ Even with many processors, speedup becomes saturated (no major gain).	➤ Speedup becomes almost linear as processors increase.
➤ Emphasizes the bottleneck effect of serial computation.	➤ Emphasizes that parallel computation dominates in large systems.
➤ Gives a theoretical upper bound on speedup.	➤ Gives a practical and realistic speedup model.
➤ Suitable for small or fixed-size problems.	➤ Suitable for large-scale and scientific computing problems.
➤ Suggests adding processors has limited benefit.	➤ Suggests adding processors is highly beneficial for bigger workloads.
